02_framework_evaluation

September 23, 2026

[1]:
```
# ============================================================================
# Für Spark Teil 1: Java 17 (erforderlich für PySpark 4.x) automatisch laden
# ============================================================================
!python.exe -m pip install pyspark install-jdk

import os
import jdk
from pathlib import Path

print("Prüfe lokale Java 17 Installation...")
# Java 17 fest
JAVA_17_PATH = str(Path(os.path.expanduser("~")) / ".jdk" / "jdk-17.0.20.1+1")

# Falls Java 17 noch nicht existiert, lokales Zip/Paket laden
if not Path(JAVA_17_PATH).exists():
    print("Lade kompatibles Java 17 für PySpark herunter (ca. 50MB)...")
    jdk.install('17')

# Sucht den echten, frisch installierten Java 17 Ordner auf der Festplatte
base_jdk_path = Path(os.path.expanduser("~")) / ".jdk"
java_found_path = None
for item in base_jdk_path.iterdir():
    # Stellt sicher, dass es eine Version 17 gibt.
    if item.is_dir() and "jdk-17" in item.name and (item / "bin").exists():
        java_found_path = str(item)
        break

if java_found_path:
    os.environ["JAVA_HOME"] = java_found_path
    os.environ["PATH"] = java_found_path + "\\bin;" + os.environ.get("PATH", "")
    print("=" * 70)
    print("ERFOLG: Passendes Java 17 wurde im User-Ordner aktiviert!")
    print(f"Pfad registriert: {os.environ['JAVA_HOME']}")
    print(f"Überprüfung (Muss True sein): {os.path.exists(os.
 ↪environ['JAVA_HOME'])}")
    print("=" * 70)
else:
```

```
    print("FEHLER: Java 17 Ordner wurde nicht gefunden.")
```

```
Requirement already satisfied: pyspark in C:\0_DA\Python312\Lib\site-packages
(4.2.0)
Requirement already satisfied: install-jdk in C:\0_DA\Python312\Lib\site-
packages (1.1.0)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in
C:\0_DA\Python312\Lib\site-packages (from pyspark) (0.10.9.9)
Prüfe lokale Java 17 Installation…
======================================================================
ERFOLG: Passendes Java 17 wurde im User-Ordner aktiviert!
Pfad registriert: C:\Users\alfa\.jdk\jdk-17.0.20.1+1
Überprüfung (Muss True sein): True
======================================================================
```

1 02 – Framework-Evaluierung

Dieses Notebook untersucht die praktische Verarbeitung des Projektdatensatzes mit Pandas, Dask und PySpark. Verglichen werden verschiedene Methoden und Dateiformate unter lokalen Ausführungsbedingungen.

Die Bewertung basiert auf gemessenen technischen Kennzahlen. Dazu gehören insbesondere:

- Ladezeit,
- Berechnungszeit,
- erkannte Zeilen- und Spaltenanzahl,
- Anzahl der Partitionen beziehungsweise Chunks,
- Verarbeitungsstatus,
- Python-Speicher-Peak,
- gemessener Daten- beziehungsweise Framework-Speicher.

Zusätzlich werden aufgetretene Fehler dokumentiert und hinsichtlich ihrer praktischen Bedeutung eingeordnet.

Die Untersuchung stellt keinen vollständig standardisierten Benchmark dar. Die Ergebnisse dienen als vergleichende technische Voranalyse für die Auswahl einer geeigneten Verarbeitungsstrategie im weiteren Projektverlauf.

1.1 Technischer Einrichtungsaufwand

Die Verwendung von PySpark erforderte zunächst die Einrichtung einer kompatiblen Java-Version sowie die Konfiguration von Umgebungsvariablen wie `JAVA_HOME` und `PATH`.

Zusätzlich müssen PySpark und weitere Abhängigkeiten installiert und versionskompatibel eingerichtet werden. Die Initialisierung war daher deutlich aufwendiger als die Verwendung von Pandas.

Dieser Aufwand wird bei der Bewertung berücksichtigt, auch wenn er nicht vollständig als reine Ladezeit messbar ist.

Hinweis zur Reproduzierbarkeit: Sollte dieses Notebook auf einer anderen Systemumgebung ausgeführt werden, stellt die automatisierte Initialisierungszelle die Verfügbarkeit der Java-Laufzeitumgebung sowie der erforderlichen Python-Pakete sicher.

1.2 Qualitative Vorteile von PySpark

Neben den gemessenen Ladezeiten bietet PySpark zusätzliche Funktionen, die in diesem Notebook nicht vollständig benchmarkartig bewertet werden.

Dazu gehören insbesondere:

- parallele Verarbeitung großer Datenmengen,
- Verarbeitung mehrerer Dateien als gemeinsamer Datenquelle,
- SQL-Abfragen über temporäre Views,
- komplexe Transformationen mit Spark SQL,
- mögliche spätere Übertragung auf Cluster-Umgebungen.

Beispielsweise können DataFrames als temporäre Views registriert und anschließend mit SQL-Abfragen analysiert werden.

Die ZIP-Datei ist als direkte Eingabe für PySpark nicht geeignet. PySpark interpretiert das ZIP-Archiv nicht automatisch als CSV-Datei, sondern liest den komprimierten Archivinhalt beziehungsweise erkennt keine erwartete Tabellenstruktur.

Daher wird für PySpark zusätzlich eine temporäre BZ2- oder Parquet-Datei erstellt. Diese Dateien dienen ausschließlich dem Framework-Vergleich und werden nach Abschluss der Analyse wieder entfernt.

[2]:
```
from pathlib import Path
import sys
import time
import gc
import os
import platform

import pandas as pd
import psutil

import bz2
import shutil
import zipfile
import tracemalloc
import ctypes

from project_config import (
    DATA_DIR,
    ORIGINAL_DS_ZIP_PATH,
    ORIGINAL_DS_BZ2_PATH,
    ORIGINAL_DS_PARQUET_PATH,
    ORIGINAL_DS_CSV_PATH,
```

```
    FRAMEWORK_RESULTS,
    get_project_relative_path,

)
```

[3]:
```
paths_to_check = {
    "ZIP": ORIGINAL_DS_ZIP_PATH,
    "BZ2": ORIGINAL_DS_BZ2_PATH,
    "Parquet": ORIGINAL_DS_PARQUET_PATH,
    "CSV": ORIGINAL_DS_CSV_PATH,
}
```

[4]:
```
paths_to_check_relative_files = {
    format_name: get_project_relative_path(path)
    for format_name, path in paths_to_check.items()
}
```

[5]:
```
def get_current_memory():
    """Gibt den aktuell verbrauchten RAM des Prozesses in MB zurück."""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / (1024 * 1024)
```

[6]:
```
def get_os_peak_memory_mb():
    """Gibt den maximalen Spitzenverbrauch (Peak) des Prozesses laut Windows in␣
 ↪MB zurück."""
    process = psutil.Process(os.getpid())

    # Unter Windows: historisch höchste RAM-Ausschlag im Task-Manager
    if os.name == 'nt':
        return process.memory_info().peak_wset / (1024 * 1024)
    else:
        # Fallback für Linux (Max RSS)
        import resource
        return resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / 1024
```

[7]:
```
def clean_memory():
    """Zwingt Python und C-Erweiterungen, RAM sofort freizugeben."""
    gc.collect()
    try:
        # WINDOWS: Zwingt Windows, das Working Set des Prozesses sofort zu␣
 ↪minimieren
        # Vertreibt ungenutzten "Verdachts-Speicher" sofort aus dem RAM
        ctypes.windll.kernel32.SetProcessWorkingSetSize(-1, -1, -1)
    except Exception:
        pass
```

[8]:
```
def print_top_memory_allocations(snapshot, limit=3):
    """Analysiert einen tracemalloc-Snapshot und zeigt die Top-Speicherfresser im Code."""
    snapshot = snapshot.filter_traces((
        tracemalloc.Filter(False, "<frozen importlib._bootstrap>"),
        tracemalloc.Filter(False, "<unknown>"),
    ))
    top_stats = snapshot.statistics('lineno')

    print("\n--- TOP SPEICHERFRESSER IM CODE ---")
    for index, stat in enumerate(top_stats[:limit], 1):
        frame = stat.traceback[0]
        print(f"#{index}: {frame.filename}:{frame.lineno}: {stat.size / (1024**2):.2f} MB")
```

1.3 Erstellung der Testdateien

Aus der vorhandenen ZIP-Ausgangsdatei werden einmalig alle für die Framework-Evaluierung benötigten Dateiformate erstellt.

Die Zielpfade werden ausschließlich aus `project_config.py` übernommen. Dadurch verwenden alle nachfolgenden Tests dieselben Dateien und dieselben zentral definierten Speicherorte.

Das Notebook erstellt keine zusätzlichen temporären Verzeichnisse. Die erzeugten Testdateien werden am Ende der Evaluierung ebenfalls über die Pfaddefinitionen aus `project_config.py` entfernt.

[9]:
```
file_status = []

for file_type, path in paths_to_check.items():
    path = Path(path)

    file_status.append({
        "Format": file_type,
        "Pfad": str(path),
        "Vorhanden": path.exists(),
        "Dateigröße_MB": round(path.stat().st_size / (1024 ** 2), 2)
        if path.exists()
        else None
    })

file_status_df = pd.DataFrame(file_status)
display(file_status_df)
```

```
    Format                                                Pfad  Vorhanden  \
0      ZIP  C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtr…        True
1      BZ2  C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtr…       False
2  Parquet  C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtr…       False
3      CSV  C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtr…       False
```

```
   Dateigröße_MB
0         103.13
1            NaN
2            NaN
3            NaN
```

[10]:
```python
if not ORIGINAL_DS_ZIP_PATH.exists():
    raise FileNotFoundError(
        f"ZIP-Datei nicht gefunden: {ORIGINAL_DS_ZIP_PATH}"
    )

ORIGINAL_DS_CSV_PATH.parent.mkdir(parents=True, exist_ok=True)

with zipfile.ZipFile(ORIGINAL_DS_ZIP_PATH, "r") as zip_file:
    csv_members = [
        member for member in zip_file.infolist()
        if not member.is_dir()
        and member.filename.lower().endswith(".csv")
    ]

    if not csv_members:
        raise FileNotFoundError(
            "Im ZIP-Archiv wurde keine CSV-Datei gefunden."
        )

    csv_member = csv_members[0]

    with zip_file.open(csv_member) as source_file:
        with open(ORIGINAL_DS_CSV_PATH, "wb") as target_file:
            shutil.copyfileobj(source_file, target_file)

with open(ORIGINAL_DS_CSV_PATH, "rb") as source_file:
    with bz2.open(ORIGINAL_DS_BZ2_PATH, "wb") as target_file:
        shutil.copyfileobj(source_file, target_file)

data_for_parquet = pd.read_csv(ORIGINAL_DS_CSV_PATH)

data_for_parquet.to_parquet(
    ORIGINAL_DS_PARQUET_PATH,
    engine="pyarrow",
    compression="snappy",
    index=False
)

del data_for_parquet

print("Testdateien wurden erstellt:")
```

```
print(f"CSV:     {ORIGINAL_DS_CSV_PATH}")
print(f"BZ2:     {ORIGINAL_DS_BZ2_PATH}")
print(f"Parquet: {ORIGINAL_DS_PARQUET_PATH}")
```

```
Testdateien wurden erstellt:
CSV:     C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtrainrides.csv
BZ2:     C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtrainrides.bz2
Parquet: C:\0_DA\Source\Projekt_Deutsche_Bahn\data\DBtrainrides.parquet
```

1.4 Vorbereitung der Ergebnisstruktur

Bevor die Frameworks getestet werden, wird die zentrale Struktur für die Ergebnisse vorbereitet.

Alle Messwerte werden an den in `project_config.py` definierten Speicherorten abgelegt. Dadurch bleiben die Ergebnisse einheitlich organisiert und können von den nachfolgenden Notebook-Abschnitten wiederverwendet werden.

[11]:
```
# ============================================================================
# Zentrale Ergebnisstrukturen initialisieren
# ============================================================================

benchmark_results = []

print("Ergebnisstruktur aus project_config.py wurde übernommen.")
print(f"Ergebnisdatei: {FRAMEWORK_RESULTS}")
```

```
Ergebnisstruktur aus project_config.py wurde übernommen.
Ergebnisdatei: C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\02_results.parquet
```

1.5 Untersuchungsaufbau

Die Evaluierung wird schrittweise nach Framework und Verarbeitungsmethode durchgeführt. Für alle Tests werden dieselben vorbereiteten Eingangsdaten und zentral definierten Dateipfade verwendet.

Untersucht werden:

- Pandas mit direktem Laden,
- Pandas mit Chunking,
- Dask mit direkter Berechnung,
- Dask mit zwischengespeicherten Daten,
- PySpark mit direkter SQL-Berechnung,
- PySpark mit zwischengespeicherten Daten.

Je nach Methode werden Lade- und Berechnungszeiten getrennt erfasst. Zusätzlich werden die erkannte Datenstruktur, die Anzahl der Chunks oder Partitionen, der Verarbeitungsstatus sowie verschiedene Speicherkennzahlen dokumentiert.

Die Ergebnisse aller Einzeltests werden anschließend in einer gemeinsamen Tabelle zusammengeführt. Die spätere Bewertung basiert auf diesen Messwerten und nicht auf einer vorher festgelegten qualitativen Punkteskala.

1.5.1 Einheitliche Berechnungsaufgabe

Um die Frameworks vergleichbar zu machen, wird nach dem Einlesen jeweils dieselbe fachliche Berechnungsaufgabe ausgeführt. Die konkrete technische Umsetzung wird an die Arbeitsweise des jeweiligen Frameworks angepasst.

Die Berechnung umfasst folgende Schritte:

1. Ausschluss von Datensätzen ohne gültigen Wert in `state`,
2. Filterung auf die Kategorien 3, 4 und 5,
3. Gruppierung nach `state`,
4. Ermittlung der Anzahl der Datensätze,
5. Berechnung der durchschnittlichen Abfahrtsverspätung,
6. Ermittlung der maximalen Abfahrtsverspätung,
7. Sortierung nach der durchschnittlichen Abfahrtsverspätung,
8. Auswahl der fünf führenden Ergebnisse.

Das fachliche Ergebnis bleibt somit frameworkübergreifend gleich. Unterschiede zwischen den Tests entstehen ausschließlich durch die jeweilige technische Umsetzung, das Dateiformat, die Verarbeitungsweise und den Ressourcenbedarf des Frameworks.

1.5.2 Pandas

Pandas: direktes Laden Zunächst wird untersucht, wie Pandas die vier vorbereiteten Dateiformate direkt einlesen kann.

Nach dem Einlesen wird die einheitlich definierte Berechnungsaufgabe mit Pandas-DataFrame-Operationen umgesetzt. Die Ladezeit und die Zeit für diese Berechnung werden getrennt erfasst.

[12]:
```python
def measure_pandas_direct(file_type, file_path):
    """
    Misst das direkte Einlesen UND die sofortige Berechnung mit Pandas.
    """
    # Startbedingungen auf Null setzen
    clean_memory()

    df = None
    tracemalloc.start()
    try:
        # --- 1. LADEVORGANG ---
        start_load = time.perf_counter()

        if file_type == "CSV":
            df = pd.read_csv(file_path)
        elif file_type == "BZ2":
            df = pd.read_csv(file_path, compression="bz2")
        elif file_type == "ZIP":
            df = pd.read_csv(file_path, compression="zip")
        elif file_type == "Parquet":
            df = pd.read_parquet(file_path, engine="pyarrow")
```

```
        else:
            raise ValueError(f"Unbekanntes Dateiformat: {file_type}")

        load_time = time.perf_counter() - start_load

        # Lade-Metriken erfassen
        _, peak_bytes_load = tracemalloc.get_traced_memory()
        ram_python_peak_mb = peak_bytes_load / (1024 * 1024)
        ram_data_netto_mb = df.memory_usage(deep=True).sum() / (1024 * 1024)

        # Tracemalloc-Zähler für die Berechnung zurücksetzen, um isolierten␣
↪Rechen-Peak zu messen
        tracemalloc.reset_peak()

        # --- 2. BERECHNUNGSVORGANG (Deine Chaining-Logik) ---
        start_calc = time.perf_counter()

        pandas_result = (
            df[(df["state"].notnull()) & (df["category"].isin([3, 4, 5]))]
            .groupby("state")["departure_delay_m"]
            .agg(["count", "mean", "max"])
            .reset_index()
        )

        # Spalten umbenennen
        pandas_result.columns = ["state", "anzahl_zuege",␣
↪"avg_abfahrts_verspaetung", "max_verspaetung"]

        # Sortieren und Top 5 extrahieren
        pandas_result = pandas_result.
↪sort_values(by="avg_abfahrts_verspaetung", ascending=False).head(5)

        calc_time = time.perf_counter() - start_calc

        # Berechnungs-Metriken erfassen
        _, peak_bytes_calc = tracemalloc.get_traced_memory()
        ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        result_text = (
            f"{len(df.columns)} Spalten, "
            f"{len(df)} Zeilen"
        )

        row = {
            "Framework": "Pandas",
            "Methode": "direkt",
            "Format": file_type,
```

```
            "Kompatibilität": "geeignet",
            "Ladezeit (s)": round(load_time, 6),
            "Berechnungszeit (s)": round(calc_time, 6),
            "Zeilen": len(df),
            "Spalten": len(df.columns),
            "Status": "erfolgreich",
            "Ergebnis": result_text,
            "Fehler": None,
            "RAM_Data_Netto_MB": round(ram_data_netto_mb, 2),
            "RAM_Python_Peak_MB": round(ram_python_peak_mb, 2),
            "RAM_Python_Calc_Peak_MB": round(ram_python_calc_peak_mb, 2)
        }

    except Exception as error:
        row = {
            "Framework": "Pandas",
            "Methode": "direkt",
            "Format": file_type,
            "Kompatibilität": "Fehler",
            "Ladezeit (s)": None,
            "Berechnungszeit (s)": None,
            "Zeilen": None,
            "Spalten": None,
            "Status": "fehlgeschlagen",
            "Ergebnis": "Einlesen oder Berechnung nicht möglich",
            "Fehler": repr(error),
            "RAM_Data_Netto_MB": None,
            "RAM_Python_Peak_MB": None,
            "RAM_Python_Calc_Peak_MB": None
        }

    finally:
        tracemalloc.stop()
        if df is not None:
            del df
        clean_memory()

    return row
```

[13]:
```
pandas_direct_results = []
for file_type, file_path in paths_to_check.items():
    print(f"Teste Pandas mit {file_type} ...")

    clean_memory()

    result = measure_pandas_direct(
        file_type=file_type,
```

```
        file_path=file_path
    )

    pandas_direct_results.append(result)

pandas_direct_df = pd.DataFrame(pandas_direct_results)

display(pandas_direct_df)
```

```
Teste Pandas mit ZIP …
Teste Pandas mit BZ2 …
Teste Pandas mit Parquet …
Teste Pandas mit CSV …
```

	Framework	Methode	Format	Kompatibilität	Ladezeit (s)	Berechnungszeit (s)	Zeilen	Spalten	Status	Ergebnis	Fehler	RAM_Data_Netto_MB	RAM_Python_Peak_MB	RAM_Python_Calc_Peak_MB
0	Pandas	direkt	ZIP	geeignet	23.035550	0.795007	2061357	20	erfolgreich	20 Spalten, 2061357 Zeilen	None	947.39	777.55	251.61
1	Pandas	direkt	BZ2	geeignet	36.412793	0.745345	2061357	20	erfolgreich	20 Spalten, 2061357 Zeilen	None	947.39	777.55	251.56
2	Pandas	direkt	Parquet	geeignet	0.515125	0.868837	2061357	20	erfolgreich	20 Spalten, 2061357 Zeilen	None	947.39	127.74	141.99
3	Pandas	direkt	CSV	geeignet	20.954767	0.647345	2061357	20	erfolgreich	20 Spalten, 2061357 Zeilen	None	947.39	777.53	251.56

[14]:
```
benchmark_results.extend(pandas_direct_results)

benchmark_results_df = pd.DataFrame(benchmark_results)

display(benchmark_results_df)
```

	Framework	Methode	Format	Kompatibilität	Ladezeit (s)
0	Pandas	direkt	ZIP	geeignet	23.035550
1	Pandas	direkt	BZ2	geeignet	36.412793
2	Pandas	direkt	Parquet	geeignet	0.515125
3	Pandas	direkt	CSV	geeignet	20.954767

```
   Berechnungszeit (s)   Zeilen  Spalten       Status  \
0             0.795007  2061357       20  erfolgreich
1             0.745345  2061357       20  erfolgreich
2             0.868837  2061357       20  erfolgreich
3             0.647345  2061357       20  erfolgreich

                    Ergebnis Fehler  RAM_Data_Netto_MB  RAM_Python_Peak_MB  \
0  20 Spalten, 2061357 Zeilen   None             947.39              777.55
1  20 Spalten, 2061357 Zeilen   None             947.39              777.55
2  20 Spalten, 2061357 Zeilen   None             947.39              127.74
3  20 Spalten, 2061357 Zeilen   None             947.39              777.53

   RAM_Python_Calc_Peak_MB
0                   251.61
1                   251.56
2                   141.99
3                   251.56
```

[15]:
```python
benchmark_results_df.to_parquet(
    FRAMEWORK_RESULTS,
    engine="pyarrow",
    compression="snappy",
    index=False
)

print(f"Ergebnisse gespeichert unter:")
print(FRAMEWORK_RESULTS)
```

```
Ergebnisse gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\02_results.parquet
```

Pandas: Chunking Beim Chunking wird dieselbe Berechnungsaufgabe schrittweise auf einzelnen Datenblöcken ausgeführt.

Die Filterung und die lokale Aggregation erfolgen je Chunk. Anschließend werden die Zwischenergebnisse zusammengeführt, sodass das finale Ergebnis fachlich der Berechnung des direkten Pandas-Verfahrens entspricht.

[16]:
```python
def measure_pandas_chunking(file_type, file_path, chunksize=100_000):
    """Misst das chunkweise Einlesen UND die sofortige Aggregation mit Pandas␣
 ↪(echtes Chunk-Processing)."""

    # Parquet ist bei Pandas-Chunking nicht anwendbar
    if file_type == "Parquet":
        return {
            "Framework": "Pandas",
            "Methode": "Chunking",
```

```
            "Format": file_type,
            "Kompatibilität": "nicht anwendbar",
            "Ladezeit (s)": None,
            "Berechnungszeit (s)": None,
            "Zeilen": None,
            "Spalten": None,
            "Status": "übersprungen",
            "Ergebnis": "Pandas read_parquet unterstützt kein chunksize",
            "Fehler": None,
            "RAM_Data_Netto_MB": None,
            "RAM_Python_Peak_MB": None,
            "RAM_Python_Calc_Peak_MB": None,
        }

    # 1. Startbedingungen nullen
    clean_memory()

    start_total = time.perf_counter()
    aggregated_chunks = []
    chunk_count = 0
    total_rows_processed = 0
    anzahl_spalten = None

    tracemalloc.start()

    try:
        # Iterator vorbereiten
        read_kwargs = {
            "filepath_or_buffer": file_path,
            "chunksize": chunksize,
        }

        if file_type == "BZ2":
            read_kwargs["compression"] = "bz2"
        elif file_type == "ZIP":
            read_kwargs["compression"] = "zip"
        elif file_type != "CSV":
            raise ValueError(f"Unbekanntes Dateiformat: {file_type}")

        # --- 1. LADE- UND RECHENPHASE (Pro Chunk) ---
        start_process = time.perf_counter()

        for chunk in pd.read_csv(**read_kwargs):
            if anzahl_spalten is None:
                anzahl_spalten = len(chunk.columns)

            total_rows_processed += len(chunk)
```

```
            # ECHTES CHUNKING: Filter und Groupby direkt auf dem kleinen Chunk␣
↪ausführen!
            chunk_filtered = chunk[
                (chunk["state"].notnull()) & (chunk["category"].isin([3, 4, 5]))
            ]

            # Lokale Zwischen-Aggregation pro Chunk
            chunk_grouped = (
                chunk_filtered.groupby("state")["departure_delay_m"]
                .agg(["count", "sum", "max"])  # sum statt mean, um es später␣
↪global korrekt zu berechnen
                .reset_index()
            )

            aggregated_chunks.append(chunk_grouped)
            chunk_count += 1

        load_time = time.perf_counter() - start_process

        # Lade-Peak erfassen (Speicher bis zum Ende der Chunk-Schleife)
        _, peak_bytes_load = tracemalloc.get_traced_memory()
        ram_python_peak_mb = peak_bytes_load / (1024 * 1024)

        # für die finale Berechnung nullen
        tracemalloc.reset_peak()

        # --- 2. FINALE GLOBALE AGGREGATION ---
        start_final_calc = time.perf_counter()

        # Die kleinen, voraggregierten Chunks zusammenführen
        final_interim = pd.concat(aggregated_chunks, ignore_index=True)

        # Globales Zusammenfassen der Zwischenergebnisse
        pandas_result = (
            final_interim.groupby("state")
            .agg({"count": "sum", "sum": "sum", "max": "max"})
            .reset_index()
        )

        # Durchschnitt berechnen (Summe der Verspätungen / Anzahl Züge)
        pandas_result["mean"] = pandas_result["sum"] / pandas_result["count"]

        # Finale Spaltenstruktur aufbauen und Top 5 extrahieren
        pandas_result = pandas_result[
            ["state", "count", "mean", "max"]
        ].copy()
```

```
        pandas_result.columns = [
            "state",
            "anzahl_zuege",
            "avg_abfahrts_verspaetung",
            "max_verspaetung",
        ]
        pandas_result = pandas_result.sort_values(
            by="avg_abfahrts_verspaetung", ascending=False
        ).head(5)

        calc_time = time.perf_counter() - start_final_calc

        # Berechnungs-Peak erfassen
        _, peak_bytes_calc = tracemalloc.get_traced_memory()
        ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        # Da am Ende kein Riesen-DF im RAM liegt, ist der verbleibende␣
↪Netto-Daten-RAM
        # nur die Größe des winzigen Top-5-Ergebnis-DataFrames (quasi 0 MB)
        ram_data_netto_mb = (
            pandas_result.memory_usage(deep=True).sum() / (1024 * 1024)
        )

        result_text = f"{anzahl_spalten} Spalten, {total_rows_processed}␣
↪Zeilen, {chunk_count} Chunks"

        result = {
            "Framework": "Pandas",
            "Methode": "Chunking",
            "Format": file_type,
            "Kompatibilität": "geeignet",
            "Ladezeit (s)": round(load_time, 6),
            "Berechnungszeit (s)": round(calc_time, 6),
            "Zeilen": total_rows_processed,
            "Spalten": anzahl_spalten,
            "Status": "erfolgreich",
            "Ergebnis": result_text,
            "Fehler": None,
            "RAM_Data_Netto_MB": round(ram_data_netto_mb, 4),  # Wird extrem␣
↪klein sein
            "RAM_Python_Peak_MB": round(ram_python_peak_mb, 2),
            "RAM_Python_Calc_Peak_MB": round(ram_python_calc_peak_mb, 2),
        }

    except Exception as error:
        result = {
            "Framework": "Pandas",
```

```
            "Methode": "Chunking",
            "Format": file_type,
            "Kompatibilität": "Fehler",
            "Ladezeit (s)": None,
            "Berechnungszeit (s)": None,
            "Zeilen": None,
            "Spalten": None,
            "Status": "fehlgeschlagen",
            "Ergebnis": "Chunkweises Einlesen oder Berechnung fehlgeschlagen",
            "Fehler": repr(error),
            "RAM_Data_Netto_MB": None,
            "RAM_Python_Peak_MB": None,
            "RAM_Python_Calc_Peak_MB": None,
        }

    finally:
        tracemalloc.stop()
        if "aggregated_chunks" in locals():
            del aggregated_chunks
        if "final_interim" in locals():
            del final_interim
        clean_memory()

    return result
```

[17]:
```
pandas_chunking_results = []

for file_type, file_path in paths_to_check.items():
    print(f"Teste Pandas-Chunking mit {file_type} ...")

    clean_memory()

    result = measure_pandas_chunking(
        file_type=file_type,
        file_path=file_path,
        chunksize=100_000
    )

    pandas_chunking_results.append(result)

pandas_chunking_df = pd.DataFrame(pandas_chunking_results)

display(pandas_chunking_df)
```

```
Teste Pandas-Chunking mit ZIP …
Teste Pandas-Chunking mit BZ2 …
Teste Pandas-Chunking mit Parquet …
Teste Pandas-Chunking mit CSV …
```

```
  Framework   Methode   Format   Kompatibilität  Ladezeit (s)  \
0    Pandas  Chunking      ZIP         geeignet     23.596085
1    Pandas  Chunking      BZ2         geeignet     37.948140
2    Pandas  Chunking  Parquet  nicht anwendbar           NaN
3    Pandas  Chunking      CSV         geeignet     20.912869

   Berechnungszeit (s)     Zeilen  Spalten        Status  \
0             0.011319  2061357.0     20.0   erfolgreich
1             0.010420  2061357.0     20.0   erfolgreich
2                  NaN        NaN      NaN  übersprungen
3             0.011362  2061357.0     20.0   erfolgreich

                                      Ergebnis Fehler  RAM_Data_Netto_MB  \
0           20 Spalten, 2061357 Zeilen, 21 Chunks   None             0.0003
1           20 Spalten, 2061357 Zeilen, 21 Chunks   None             0.0003
2  Pandas read_parquet unterstützt kein chunksize   None                NaN
3           20 Spalten, 2061357 Zeilen, 21 Chunks   None             0.0003

   RAM_Python_Peak_MB  RAM_Python_Calc_Peak_MB
0               50.61                     6.88
1               49.91                     6.87
2                 NaN                      NaN
3               50.34                     6.88
```

[18]:
```
benchmark_results.extend(pandas_chunking_results)

benchmark_results_df = pd.DataFrame(benchmark_results)

display(benchmark_results_df)
```

```
  Framework   Methode   Format   Kompatibilität  Ladezeit (s)  \
0    Pandas    direkt      ZIP         geeignet     23.035550
1    Pandas    direkt      BZ2         geeignet     36.412793
2    Pandas    direkt  Parquet         geeignet      0.515125
3    Pandas    direkt      CSV         geeignet     20.954767
4    Pandas  Chunking      ZIP         geeignet     23.596085
5    Pandas  Chunking      BZ2         geeignet     37.948140
6    Pandas  Chunking  Parquet  nicht anwendbar           NaN
7    Pandas  Chunking      CSV         geeignet     20.912869

   Berechnungszeit (s)     Zeilen  Spalten        Status  \
0             0.795007  2061357.0     20.0   erfolgreich
1             0.745345  2061357.0     20.0   erfolgreich
2             0.868837  2061357.0     20.0   erfolgreich
3             0.647345  2061357.0     20.0   erfolgreich
4             0.011319  2061357.0     20.0   erfolgreich
5             0.010420  2061357.0     20.0   erfolgreich
6                  NaN        NaN      NaN  übersprungen
```

```
7             0.011362  2061357.0     20.0   erfolgreich

                                        Ergebnis Fehler  RAM_Data_Netto_MB  \
0                     20 Spalten, 2061357 Zeilen   None           947.3900
1                     20 Spalten, 2061357 Zeilen   None           947.3900
2                     20 Spalten, 2061357 Zeilen   None           947.3900
3                     20 Spalten, 2061357 Zeilen   None           947.3900
4           20 Spalten, 2061357 Zeilen, 21 Chunks   None             0.0003
5           20 Spalten, 2061357 Zeilen, 21 Chunks   None             0.0003
6  Pandas read_parquet unterstützt kein chunksize   None                NaN
7           20 Spalten, 2061357 Zeilen, 21 Chunks   None             0.0003

   RAM_Python_Peak_MB  RAM_Python_Calc_Peak_MB
0              777.55                   251.61
1              777.55                   251.56
2              127.74                   141.99
3              777.53                   251.56
4               50.61                     6.88
5               49.91                     6.87
6                 NaN                      NaN
7               50.34                     6.88
```

[19]:
```
benchmark_results_df.to_parquet(
    FRAMEWORK_RESULTS,
    engine="pyarrow",
    compression="snappy",
    index=False
)

print(f"Ergebnisse gespeichert unter: {FRAMEWORK_RESULTS}")
```

```
Ergebnisse gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\02_results.parquet
```

1.5.3 Dask

Dask ist ein Framework für parallele und verteilte Verarbeitung größerer Datenmengen. Ein Dask-DataFrame besteht aus mehreren partitionierten Pandas-DataFrames.

Dask eignet sich besonders für große Dateien oder mehrere Dateien, die nicht vollständig in den Arbeitsspeicher geladen werden sollen.

Dask setzt dieselbe Berechnungsaufgabe mit verzögerten DataFrame-Operationen um. Filterung, Gruppierung und Aggregation werden dabei auf den Dask- Partitionen geplant und anschließend durch `.compute()` ausgeführt.

Die Berechnungszeit beschreibt ausschließlich die Ausführung dieser einheitlichen Berechnungsaufgabe.

[20]:
```
!pip install "dask[complete]"
```

```
Requirement already satisfied: dask[complete] in C:\0_DA\Python312\Lib\site-
packages (2026.8.0)
Requirement already satisfied: click>=8.1 in C:\0_DA\Python312\Lib\site-packages
(from dask[complete]) (8.5.0)
Requirement already satisfied: cloudpickle>=3.0.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (3.1.2)
Requirement already satisfied: fsspec>=2021.09.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (2026.7.0)
Requirement already satisfied: packaging>=20.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (26.3)
Requirement already satisfied: partd>=1.4.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (1.4.2)
Requirement already satisfied: pyyaml>=5.4.1 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (6.0.3)
Requirement already satisfied: toolz>=0.12.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (1.1.0)
Requirement already satisfied: lz4>=4.3.2 in C:\0_DA\Python312\Lib\site-packages
(from dask[complete]) (4.4.5)
Requirement already satisfied: locket in C:\0_DA\Python312\Lib\site-packages
(from partd>=1.4.0->dask[complete]) (1.0.0)
Requirement already satisfied: numpy>=1.24 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (2.5.2)
Requirement already satisfied: pandas>=2.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (3.0.5)
Requirement already satisfied: pyarrow>=16.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (25.0.1)
Requirement already satisfied: distributed<2026.8.1,>=2026.8.0 in
C:\0_DA\Python312\Lib\site-packages (from dask[complete]) (2026.8.0)
Requirement already satisfied: bokeh>=3.1.0 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (3.10.0)
Requirement already satisfied: jinja2>=2.10.3 in C:\0_DA\Python312\Lib\site-
packages (from dask[complete]) (3.1.6)
Requirement already satisfied: msgpack>=1.0.2 in C:\0_DA\Python312\Lib\site-
packages (from distributed<2026.8.1,>=2026.8.0->dask[complete]) (1.2.2)
Requirement already satisfied: psutil>=5.8.0 in C:\0_DA\Python312\Lib\site-
packages (from distributed<2026.8.1,>=2026.8.0->dask[complete]) (7.2.2)
Requirement already satisfied: sortedcontainers>=2.0.5 in
C:\0_DA\Python312\Lib\site-packages (from
distributed<2026.8.1,>=2026.8.0->dask[complete]) (2.4.0)
Requirement already satisfied: tblib!=3.2.0,!=3.2.1,>=1.6.0 in
C:\0_DA\Python312\Lib\site-packages (from
distributed<2026.8.1,>=2026.8.0->dask[complete]) (3.2.2)
Requirement already satisfied: tornado>=6.2.0 in C:\0_DA\Python312\Lib\site-
packages (from distributed<2026.8.1,>=2026.8.0->dask[complete]) (6.5.8)
Requirement already satisfied: zict>=3.0.0 in C:\0_DA\Python312\Lib\site-
packages (from distributed<2026.8.1,>=2026.8.0->dask[complete]) (3.0.0)
Requirement already satisfied: narwhals>=1.13 in C:\0_DA\Python312\Lib\site-
packages (from bokeh>=3.1.0->dask[complete]) (2.25.0)
```

```
Requirement already satisfied: pillow>=7.1.0 in C:\0_DA\Python312\Lib\site-
packages (from bokeh>=3.1.0->dask[complete]) (12.3.0)
Requirement already satisfied: xyzservices>=2021.09.1 in
C:\0_DA\Python312\Lib\site-packages (from bokeh>=3.1.0->dask[complete])
(2026.9.1)
Requirement already satisfied: MarkupSafe>=2.0 in C:\0_DA\Python312\Lib\site-
packages (from jinja2>=2.10.3->dask[complete]) (3.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in
C:\0_DA\Python312\Lib\site-packages (from pandas>=2.0->dask[complete])
(2.9.0.post0)
Requirement already satisfied: tzdata in C:\0_DA\Python312\Lib\site-packages
(from pandas>=2.0->dask[complete]) (2026.3)
Requirement already satisfied: six>=1.5 in C:\0_DA\Python312\Lib\site-packages
(from python-dateutil>=2.8.2->pandas>=2.0->dask[complete]) (1.17.0)
```

[21]:
```
import dask.dataframe as dd
from dask.distributed import wait
from dask.distributed import Client
```

Dask direkt Die Dateien werden direkt aus ihren ursprünglichen Formaten mit Dask eingelesen. Dask erstellt dabei einen verzögerten Verarbeitungsgrafo, der erst durch `.compute()` tatsächlich ausgeführt wird.

[22]:
```
def measure_dask_direct(file_type, file_path, dask_client):
    """
    Misst die kombinierte End-to-End-Verarbeitung (Laden + Berechnen) mit Dask␣
 ↪direkt von der Festplatte.
    """
    #  Startbedingungen auf dem Hauptprozess und den Workern nullen
    clean_memory()
    dask_client.restart()

    ddf = None
    load_time = None
    ram_python_peak_mb = None
    ram_python_calc_peak_mb = None
    calc_time = None
    tracemalloc.start()

    try:
        # --- GRAPH AUFBAUEN (Lazy Loading - Kostet fast keine Zeit) ---
        start_load = time.perf_counter()
        if file_type == "CSV":
            ddf = dd.read_csv(file_path)
        elif file_type == "BZ2":
            ddf = dd.read_csv(file_path, compression="bz2", blocksize=None)
        elif file_type == "ZIP":
```

```
            ddf = dd.read_csv(file_path, compression="zip", blocksize=None)
        elif file_type == "Parquet":
            ddf = dd.read_parquet(file_path)
        else:
            raise ValueError(f"Unbekanntes Dateiformat: {file_type}")

        # Graph Aufbau
        load_time = time.perf_counter() - start_load

        _, peak_bytes = tracemalloc.get_traced_memory()
        ram_python_peak_mb = peak_bytes / (1024 * 1024)

        #  BERECHNUNGSHINWEISE UND EXECUTION ---
        start_calc = time.perf_counter()

        # Zähler zurücksetzen für den isolierten Rechen-Peak in Python
        tracemalloc.reset_peak()

        # Chaining-Logik für Dask
        dask_task = (
            ddf[(ddf["state"].notnull()) & (ddf["category"].isin([3, 4, 5]))]
            .groupby("state")["departure_delay_m"]
            .agg(["count", "mean", "max"])
            .reset_index()
        )

        dask_task = dask_task.rename(columns={
            "count": "anzahl_zuege",
            "mean": "avg_abfahrts_verspaetung",
            "max": "max_verspaetung"
        })

        # .compute() zwingt Dask, den optimierten Graphen JETZT auszuführen
        # Dask streamt die Daten, filtert sofort und gibt nur die Top 5 an␣
↪Python zurück!
        pandas_result = dask_task.nlargest(5, "avg_abfahrts_verspaetung").
↪compute()

        calc_time = time.perf_counter() - start_calc

        # Python-Skript Peaks auslesen
        _, peak_bytes_calc = tracemalloc.get_traced_memory()
        ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        # Framework-interne Datengröße (Da die Worker danach wieder leer sind,
        # ist es nur das winzige resultierende Pandas-DF mit 5 Zeilen)
```

```
        ram_data_netto_mb = pandas_result.memory_usage(deep=True).sum() / (1024␣
↪* 1024)

        rows_len = len(ddf)
        result_text = f"{len(ddf.columns)} Spalten, {rows_len} Zeilen, {ddf.
↪npartitions} Partitionen"

        row = {
            "Framework": "Dask",
            "Methode": "direkt",
            "Format": file_type,
            "Kompatibilität": "geeignet",
            "Ladezeit (s)": (None if load_time is None else round(load_time,␣
↪4)),
            "Berechnungszeit (s)": round(calc_time, 4),
            "Zeilen": rows_len,
            "Spalten": len(ddf.columns),
            "Status": "erfolgreich",
            "Ergebnis": result_text,
            "Fehler": None,
            # Die synchronisierten RAM-Spalten:
            "RAM_Data_Netto_MB": round(ram_data_netto_mb, 4),
            "RAM_Python_Peak_MB":(
                None if ram_python_peak_mb is None
                     else round(ram_python_peak_mb, 2)
            ),  # (Graph-Erstellung)
            "RAM_Python_Calc_Peak_MB": round(ram_python_calc_peak_mb, 2)
        }

    except Exception as error:
        if ram_python_peak_mb is not None:
            _, peak_bytes_calc = tracemalloc.get_traced_memory()
            ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        if load_time is not None:
            calc_time = time.perf_counter() - start_calc


        row = {
            "Framework": "Dask",
            "Methode": "direkt",
            "Format": file_type,
            "Kompatibilität": "Fehler",
            "Ladezeit (s)": (None if load_time is None else round(load_time,␣
↪6)),
            "Berechnungszeit (s)": (None if calc_time is None else␣
↪round(calc_time, 6)),
```

```
            "Zeilen": None,
            "Spalten": None,
            "Status": "fehlgeschlagen",
            "Ergebnis": "Direkte Berechnung in Dask fehlgeschlagen",
            "Fehler": repr(error),
            "RAM_Data_Netto_MB": None,
            "RAM_Python_Peak_MB":( None if ram_python_peak_mb is None
                    else round(ram_python_peak_mb, 2)
            ),
            "RAM_Python_Calc_Peak_MB": ( None if ram_python_calc_peak_mb is None
                    else round(ram_python_calc_peak_mb, 2)
            ),
        }

    finally:
        tracemalloc.stop()
        if ddf is not None:
            del ddf
        clean_memory()

    return row
```

[23]:
```
#  Dask-Client einmalig VOR der Schleife starten
# (LocalCluster wird automatisch im Hintergrund erstellt)
client = Client()

dask_direct_results = []

try:
    for file_type, file_path in paths_to_check.items():
        print(f"Dask-Test für {file_type} ...")


        # Methode aufrufen und den 'client' mitgeben
        result = measure_dask_direct(
            file_type=file_type, file_path=file_path, dask_client=client
        )

        dask_direct_results.append(result)

finally:
    # WICHTIG: Client am Ende immer schließen, damit keine Ports blockiert␣
 ↪bleiben
    client.close()

dask_direct_df = pd.DataFrame(dask_direct_results)
display(dask_direct_df)
```

```
Dask-Test für ZIP …

2026-09-23 10:51:04,274 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:04,279 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:04,288 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:04,356 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:19,467 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58233 (pid=2952) exceeded 95% memory budget. Restarting…
2026-09-23 10:51:19,632 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:32,995 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58222 (pid=7116) exceeded 95% memory budget. Restarting…
2026-09-23 10:51:33,005 - distributed.scheduler - WARNING - Removing worker
'tcp://127.0.0.1:58222' caused the cluster to lose already computed task(s),
which will be recomputed elsewhere:
{('delayed-9c22ef80a56ff25bdd0d721e703914fc', 0)} (stimulus_id='handle-worker-
cleanup-1790153493.0047123')
2026-09-23 10:51:33,142 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:46,654 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58228 (pid=9192) exceeded 95% memory budget. Restarting…
2026-09-23 10:51:46,806 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:59,479 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58250 (pid=6316) exceeded 95% memory budget. Restarting…
2026-09-23 10:51:59,485 - distributed.scheduler - ERROR - Task ('read-csv-
ad7fe2a87dab5d3bbdfff3c815a9f995', 0) marked as failed because 4 workers died
while trying to run it
2026-09-23 10:51:59,695 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:59,801 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:59,803 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:51:59,815 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)

Dask-Test für BZ2 …

2026-09-23 10:52:29,244 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58275 (pid=12728) exceeded 95% memory budget. Restarting…
2026-09-23 10:52:29,397 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:52:58,494 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58276 (pid=2340) exceeded 95% memory budget. Restarting…
```

```
2026-09-23 10:52:58,500 - distributed.scheduler - WARNING - Removing worker
'tcp://127.0.0.1:58276' caused the cluster to lose already computed task(s),
which will be recomputed elsewhere:
{('delayed-9c22ef80a56ff25bdd0d721e703914fc', 0)} (stimulus_id='handle-worker-
cleanup-1790153578.5011525')
2026-09-23 10:52:58,628 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:53:27,281 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58268 (pid=1976) exceeded 95% memory budget. Restarting…
2026-09-23 10:53:27,422 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:53:56,106 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58299 (pid=6884) exceeded 95% memory budget. Restarting…
2026-09-23 10:53:56,113 - distributed.scheduler - ERROR - Task ('read-
csv-7166cdbdd63db72df826112457892bef', 0) marked as failed because 4 workers
died while trying to run it
2026-09-23 10:53:56,338 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)

Dask-Test für Parquet …

2026-09-23 10:53:56,488 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:53:56,493 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:53:56,495 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)

Dask-Test für CSV …

2026-09-23 10:53:59,711 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:53:59,903 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:53:59,907 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:53:59,911 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
```

	Framework	Methode	Format	Kompatibilität	Ladezeit (s)	Berechnungszeit (s)	Zeilen	Spalten	Status
0	Dask	direkt	ZIP	Fehler	0.066706	53.702210	NaN	NaN	fehlgeschlagen
1	Dask	direkt	BZ2	Fehler	0.052605	115.299709	NaN	NaN	fehlgeschlagen
2	Dask	direkt	Parquet	geeignet	0.047000	1.532400	2061357.0	20.0	erfolgreich
3	Dask	direkt	CSV	geeignet	0.055500	6.257700	2061357.0	20.0	erfolgreich

```
                                    Ergebnis  \
0   Direkte Berechnung in Dask fehlgeschlagen
1   Direkte Berechnung in Dask fehlgeschlagen
2   20 Spalten, 2061357 Zeilen, 1 Partitionen
3  20 Spalten, 2061357 Zeilen, 12 Partitionen

                                              Fehler  RAM_Data_Netto_MB  \
0  KilledWorker(('read-csv-ad7fe2a87dab5d3bbdfff3…                NaN
1  KilledWorker(('read-csv-7166cdbdd63db72df82611…                NaN
2                                                NaN             0.0003
3                                                NaN             0.0003

   RAM_Python_Peak_MB  RAM_Python_Calc_Peak_MB
0                1.25                     1.56
1                1.24                     1.67
2                0.23                     1.21
3                1.25                     1.86
```

[24]:
```
benchmark_results.extend(dask_direct_results)

benchmark_results_df = pd.DataFrame(benchmark_results)

display(benchmark_results_df)
```

```
   Framework   Methode   Format   Kompatibilität  Ladezeit (s)  \
0     Pandas    direkt      ZIP         geeignet     23.035550
1     Pandas    direkt      BZ2         geeignet     36.412793
2     Pandas    direkt  Parquet         geeignet      0.515125
3     Pandas    direkt      CSV         geeignet     20.954767
4     Pandas  Chunking      ZIP         geeignet     23.596085
5     Pandas  Chunking      BZ2         geeignet     37.948140
6     Pandas  Chunking  Parquet  nicht anwendbar           NaN
7     Pandas  Chunking      CSV         geeignet     20.912869
8       Dask    direkt      ZIP           Fehler      0.066706
9       Dask    direkt      BZ2           Fehler      0.052605
10      Dask    direkt  Parquet         geeignet      0.047000
11      Dask    direkt      CSV         geeignet      0.055500

    Berechnungszeit (s)     Zeilen  Spalten          Status  \
0              0.795007  2061357.0     20.0     erfolgreich
1              0.745345  2061357.0     20.0     erfolgreich
2              0.868837  2061357.0     20.0     erfolgreich
3              0.647345  2061357.0     20.0     erfolgreich
4              0.011319  2061357.0     20.0     erfolgreich
5              0.010420  2061357.0     20.0     erfolgreich
6                   NaN        NaN      NaN    übersprungen
7              0.011362  2061357.0     20.0     erfolgreich
```

```
8             53.702210        NaN      NaN  fehlgeschlagen
9            115.299709        NaN      NaN  fehlgeschlagen
10             1.532400  2061357.0     20.0     erfolgreich
11             6.257700  2061357.0     20.0     erfolgreich

                                        Ergebnis  \
0                      20 Spalten, 2061357 Zeilen
1                      20 Spalten, 2061357 Zeilen
2                      20 Spalten, 2061357 Zeilen
3                      20 Spalten, 2061357 Zeilen
4            20 Spalten, 2061357 Zeilen, 21 Chunks
5            20 Spalten, 2061357 Zeilen, 21 Chunks
6   Pandas read_parquet unterstützt kein chunksize
7            20 Spalten, 2061357 Zeilen, 21 Chunks
8         Direkte Berechnung in Dask fehlgeschlagen
9         Direkte Berechnung in Dask fehlgeschlagen
10        20 Spalten, 2061357 Zeilen, 1 Partitionen
11       20 Spalten, 2061357 Zeilen, 12 Partitionen

                                                 Fehler  RAM_Data_Netto_MB  \
0                                                   NaN           947.3900
1                                                   NaN           947.3900
2                                                   NaN           947.3900
3                                                   NaN           947.3900
4                                                   NaN             0.0003
5                                                   NaN             0.0003
6                                                   NaN                NaN
7                                                   NaN             0.0003
8   KilledWorker(('read-csv-ad7fe2a87dab5d3bbdfff3…                 NaN
9   KilledWorker(('read-csv-7166cdbdd63db72df82611…                 NaN
10                                                  NaN             0.0003
11                                                  NaN             0.0003

    RAM_Python_Peak_MB  RAM_Python_Calc_Peak_MB
0               777.55                   251.61
1               777.55                   251.56
2               127.74                   141.99
3               777.53                   251.56
4                50.61                     6.88
5                49.91                     6.87
6                  NaN                      NaN
7                50.34                     6.88
8                 1.25                     1.56
9                 1.24                     1.67
10                0.23                     1.21
11                1.25                     1.86
```

[25]:
```
benchmark_results_df.to_parquet(
    FRAMEWORK_RESULTS,
    engine="pyarrow",
    compression="snappy",
    index=False
)

print(
    f"Ergebnisse gespeichert unter: "
    f"{FRAMEWORK_RESULTS}"
)
```

```
Ergebnisse gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\02_results.parquet
```

Dask cached Bei dieser Variante wird der mit Dask erstellte DataFrame zunächst mit `persist()` im Speicher der lokalen Dask-Worker materialisiert.

Dadurch werden die Daten vor der eigentlichen Berechnung zwischengespeichert. Die anschließende Auswertung kann somit auf den bereits materialisierten Partitionen aufbauen.

Die gemessene Ladezeit umfasst den Aufbau und die Materialisierung des Dask-DataFrames.

[26]:
```
def measure_dask_cached(file_type, file_path, dask_client):
    """
    Misst das Laden von Dask-Daten in den Worker-RAM (cached) und führt danach
    eine isolierte In-Memory-Berechnung durch.
    """
    # Startbedingungen überall nullen
    clean_memory()
    dask_client.restart()

    load_time = None
    ram_python_peak_mb = None
    ram_python_calc_peak_mb = None
    calc_time = None
    start_calc = None
    ram_data_netto_mb = None
    start_load= None

    ddf = None
    tracemalloc.start()

    try:
        # --- LADEPHASE (In den RAM der Worker laden via .persist()) ---
        start_load = time.perf_counter()

        if file_type == "CSV":
```

```
            ddf = dd.read_csv(file_path)
        elif file_type == "BZ2":
            ddf = dd.read_csv(file_path, compression="bz2", blocksize=None)
        elif file_type == "ZIP":
            ddf = dd.read_csv(file_path, compression="zip", blocksize=None)
        elif file_type == "Parquet":
            ddf = dd.read_parquet(file_path)
        else:
            raise ValueError(f"Unbekanntes Dateiformat: {file_type}")

        # Daten im RAM der Worker fixieren
        ddf = ddf.persist()
        wait(ddf) # Warten, bis das Laden im RAM abgeschlossen ist

        load_time = time.perf_counter() - start_load

        _, peak_bytes_load = tracemalloc.get_traced_memory()
        ram_python_peak_mb = peak_bytes_load / (1024 * 1024)

        # Framework-interne Netto-Datengröße im Worker-Cluster abfragen
        # .compute() berechnet hier nur die Byte-Summe der internen Arrays
        ram_data_netto_mb = ddf.memory_usage(deep=True).sum().compute() / (1024␣
↪* 1024)

        tracemalloc.reset_peak()

        start_calc = time.perf_counter()

        dask_task = (
            ddf[(ddf["state"].notnull()) & (ddf["category"].isin([3, 4, 5]))]
            .groupby("state")["departure_delay_m"]
            .agg(["count", "mean", "max"])
            .reset_index()
        )

        dask_task = dask_task.rename(columns={
            "count": "anzahl_zuege",
            "mean": "avg_abfahrts_verspaetung",
            "max": "max_verspaetung"
        })

        # Ausführen auf den bereits gecachten Daten
        pandas_result = dask_task.nlargest(5, "avg_abfahrts_verspaetung").
↪compute()

        calc_time = time.perf_counter() - start_calc
```

```
        # Berechnungs-Peak des Hauptprozesses erfassen
        _, peak_bytes_calc = tracemalloc.get_traced_memory()
        ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        rows_len = len(ddf)
        result_text = f"{len(ddf.columns)} Spalten, {rows_len} Zeilen, {ddf.
↪npartitions} Partitionen"

        row = {
            "Framework": "Dask",
            "Methode": "cached",
            "Format": file_type,
            "Kompatibilität": "geeignet",
            "Ladezeit (s)": round(load_time, 6),
            "Berechnungszeit (s)": round(calc_time, 6),
            "Zeilen": rows_len,
            "Spalten": len(ddf.columns),
            "Status": "erfolgreich",
            "Ergebnis": result_text,
            "Fehler": None,
            "RAM_Data_Netto_MB": round(ram_data_netto_mb, 2),
            "RAM_Python_Peak_MB": round(ram_python_peak_mb, 2),
            "RAM_Python_Calc_Peak_MB": round(ram_python_calc_peak_mb, 2)
        }

    except Exception as error:

        # also mit berechnung wurde angefangen
        if load_time is not None and start_calc is not None:
            calc_time = time.perf_counter() - load_calc
        # gestartet aber abgebrochen
        if start_load is not None and load_time is None:
            load_time =ime.perf_counter() - start_load

        # Also loaded
        if  ram_python_peak_mb is not None and ram_data_netto_mb is not None:
             _, peak_bytes_calc = tracemalloc.get_traced_memory()
             ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        row = {
            "Framework": "Dask",
            "Methode": "cached",
            "Format": file_type,
            "Kompatibilität": "Fehler",
            "Ladezeit (s)": (None if load_time is None else round(load_time,␣
↪6)),
```

```
            "Berechnungszeit (s)": (None if calc_time is None else␣
 ↪round(calc_time, 6)),
            "Zeilen": None,
            "Spalten": None,
            "Status": "fehlgeschlagen",
            "Ergebnis": "Cached Berechnung in Dask fehlgeschlagen",
            "Fehler": repr(error),
            "RAM_Data_Netto_MB": (None if ram_data_netto_mb is None else␣
 ↪round(ram_data_netto_mb, 2)),
            "RAM_Python_Peak_MB":( None if ram_python_peak_mb is None
                    else round(ram_python_peak_mb, 4)
            ),
            "RAM_Python_Calc_Peak_MB": ( None if ram_python_calc_peak_mb is None
                    else round(ram_python_calc_peak_mb, 4)
            ),
        }

    finally:
        tracemalloc.stop()
        if ddf is not None:
            del ddf
        clean_memory()

    return row
```

[27]:
```
# Dask-Client einmalig VOR der Schleife starten
# (LocalCluster wird automatisch im Hintergrund erstellt)
client = Client()

dask_cached_results = []

try:
    for file_type, file_path in paths_to_check.items():
        print(f"Dask-Test für {file_type} ...")

        result = measure_dask_cached(
            file_type=file_type, file_path=file_path, dask_client=client
        )

        dask_cached_results.append(result)

finally:
    # WICHTIG: Client am Ende immer schließen, damit keine Ports blockiert␣
 ↪bleiben
    client.close()

dask_cached_df = pd.DataFrame(dask_cached_results)
```

```
display(dask_cached_df)
```

```
2026-09-23 10:54:14,384 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:54:14,389 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:54:14,392 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:54:14,398 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
```

```
Dask-Test für ZIP …
```

```
2026-09-23 10:54:29,498 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58441 (pid=11060) exceeded 95% memory budget. Restarting…
2026-09-23 10:54:29,646 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:54:43,075 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58438 (pid=8936) exceeded 95% memory budget. Restarting…
2026-09-23 10:54:43,212 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:54:57,098 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58456 (pid=7808) exceeded 95% memory budget. Restarting…
2026-09-23 10:54:57,244 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:55:12,207 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58437 (pid=1916) exceeded 95% memory budget. Restarting…
2026-09-23 10:55:12,215 - distributed.scheduler - ERROR - Task ('read-
_to_string_dtype-5dec73148d9b5fbb89b9fe8213353716', 0) marked as failed because
4 workers died while trying to run it
2026-09-23 10:55:12,489 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:55:12,588 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:55:12,601 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:55:12,606 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
```

```
Dask-Test für BZ2 …
```

```
2026-09-23 10:55:44,110 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58481 (pid=10788) exceeded 95% memory budget. Restarting…
2026-09-23 10:55:44,264 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:56:12,248 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58488 (pid=5224) exceeded 95% memory budget. Restarting…
2026-09-23 10:56:12,401 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:56:43,216 - distributed.nanny.memory - WARNING - Worker
```

```
tcp://127.0.0.1:58510 (pid=4548) exceeded 95% memory budget. Restarting…
2026-09-23 10:56:43,416 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:57:12,027 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58487 (pid=8136) exceeded 95% memory budget. Restarting…
2026-09-23 10:57:12,036 - distributed.scheduler - ERROR - Task ('read-
_to_string_dtype-ceb7586fa0b959e3419b203adbb9c361', 0) marked as failed because
4 workers died while trying to run it
2026-09-23 10:57:12,294 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:57:12,421 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:57:12,430 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:57:12,454 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)

Dask-Test für Parquet …

2026-09-23 10:57:17,870 - distributed.nanny.memory - WARNING - Worker
tcp://127.0.0.1:58543 (pid=5092) exceeded 95% memory budget. Restarting…
2026-09-23 10:57:17,891 - distributed.scheduler - WARNING - Removing worker
'tcp://127.0.0.1:58543' caused the cluster to lose already computed task(s),
which will be recomputed elsewhere:
{('read_parquet-2748d2f415d080d918eedd188d3aef89', 0)} (stimulus_id='handle-
worker-cleanup-1790153837.8919199')
2026-09-23 10:57:18,051 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:57:28,604 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:57:28,621 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)
2026-09-23 10:57:28,625 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)

Dask-Test für CSV …

2026-09-23 10:57:28,940 - distributed.nanny - WARNING - Restarting worker
(status=Status.running)

  Framework Methode   Format Kompatibilität  Ladezeit (s)  \
0      Dask  cached      ZIP         Fehler     56.807416
1      Dask  cached      BZ2         Fehler    118.478661
2      Dask  cached  Parquet       geeignet      2.401278
3      Dask  cached      CSV       geeignet      4.926066

   Berechnungszeit (s)     Zeilen  Spalten          Status  \
0                  NaN        NaN      NaN  fehlgeschlagen
1                  NaN        NaN      NaN  fehlgeschlagen
2             4.676405  2061357.0     20.0     erfolgreich
```

```
3             0.413779  2061357.0     20.0     erfolgreich

                                    Ergebnis  \
0    Cached Berechnung in Dask fehlgeschlagen
1    Cached Berechnung in Dask fehlgeschlagen
2   20 Spalten, 2061357 Zeilen, 1 Partitionen
3  20 Spalten, 2061357 Zeilen, 12 Partitionen

                                             Fehler  RAM_Data_Netto_MB  \
0  KilledWorker(('read-_to_string_dtype-5dec73148…                NaN
1  KilledWorker(('read-_to_string_dtype-ceb7586fa…                NaN
2                                                NaN             947.39
3                                                NaN             947.39

   RAM_Python_Peak_MB  RAM_Python_Calc_Peak_MB
0              1.2413                      NaN
1              1.4289                      NaN
2              0.7300                     1.29
3              1.2500                     2.06
```

[28]:
```
benchmark_results.extend(dask_cached_results)

benchmark_results_df = pd.DataFrame(benchmark_results)

display(benchmark_results_df)
```

```
   Framework   Methode   Format   Kompatibilität  Ladezeit (s)  \
0     Pandas    direkt      ZIP         geeignet     23.035550
1     Pandas    direkt      BZ2         geeignet     36.412793
2     Pandas    direkt  Parquet         geeignet      0.515125
3     Pandas    direkt      CSV         geeignet     20.954767
4     Pandas  Chunking      ZIP         geeignet     23.596085
5     Pandas  Chunking      BZ2         geeignet     37.948140
6     Pandas  Chunking  Parquet  nicht anwendbar           NaN
7     Pandas  Chunking      CSV         geeignet     20.912869
8       Dask    direkt      ZIP           Fehler      0.066706
9       Dask    direkt      BZ2           Fehler      0.052605
10      Dask    direkt  Parquet         geeignet      0.047000
11      Dask    direkt      CSV         geeignet      0.055500
12      Dask    cached      ZIP           Fehler     56.807416
13      Dask    cached      BZ2           Fehler    118.478661
14      Dask    cached  Parquet         geeignet      2.401278
15      Dask    cached      CSV         geeignet      4.926066

    Berechnungszeit (s)     Zeilen  Spalten          Status  \
0              0.795007  2061357.0     20.0     erfolgreich
1              0.745345  2061357.0     20.0     erfolgreich
2              0.868837  2061357.0     20.0     erfolgreich
```

```
3              0.647345  2061357.0     20.0     erfolgreich
4              0.011319  2061357.0     20.0     erfolgreich
5              0.010420  2061357.0     20.0     erfolgreich
6                   NaN        NaN      NaN    übersprungen
7              0.011362  2061357.0     20.0     erfolgreich
8             53.702210        NaN      NaN  fehlgeschlagen
9            115.299709        NaN      NaN  fehlgeschlagen
10             1.532400  2061357.0     20.0     erfolgreich
11             6.257700  2061357.0     20.0     erfolgreich
12                  NaN        NaN      NaN  fehlgeschlagen
13                  NaN        NaN      NaN  fehlgeschlagen
14             4.676405  2061357.0     20.0     erfolgreich
15             0.413779  2061357.0     20.0     erfolgreich

                                      Ergebnis  \
0                         20 Spalten, 2061357 Zeilen
1                         20 Spalten, 2061357 Zeilen
2                         20 Spalten, 2061357 Zeilen
3                         20 Spalten, 2061357 Zeilen
4              20 Spalten, 2061357 Zeilen, 21 Chunks
5              20 Spalten, 2061357 Zeilen, 21 Chunks
6   Pandas read_parquet unterstützt kein chunksize
7              20 Spalten, 2061357 Zeilen, 21 Chunks
8          Direkte Berechnung in Dask fehlgeschlagen
9          Direkte Berechnung in Dask fehlgeschlagen
10         20 Spalten, 2061357 Zeilen, 1 Partitionen
11        20 Spalten, 2061357 Zeilen, 12 Partitionen
12          Cached Berechnung in Dask fehlgeschlagen
13          Cached Berechnung in Dask fehlgeschlagen
14         20 Spalten, 2061357 Zeilen, 1 Partitionen
15        20 Spalten, 2061357 Zeilen, 12 Partitionen

                                                Fehler  RAM_Data_Netto_MB  \
0                                                  NaN           947.3900
1                                                  NaN           947.3900
2                                                  NaN           947.3900
3                                                  NaN           947.3900
4                                                  NaN             0.0003
5                                                  NaN             0.0003
6                                                  NaN                NaN
7                                                  NaN             0.0003
8   KilledWorker(('read-csv-ad7fe2a87dab5d3bbdfff3…                NaN
9   KilledWorker(('read-csv-7166cdbdd63db72df82611…                NaN
10                                                 NaN             0.0003
11                                                 NaN             0.0003
12  KilledWorker(('read-_to_string_dtype-5dec73148…                NaN
13  KilledWorker(('read-_to_string_dtype-ceb7586fa…                NaN
14                                                 NaN           947.3900
```

```
15                                                  NaN           947.3900
```

```
    RAM_Python_Peak_MB  RAM_Python_Calc_Peak_MB
0             777.5500                  251.61
1             777.5500                  251.56
2             127.7400                  141.99
3             777.5300                  251.56
4              50.6100                    6.88
5              49.9100                    6.87
6                  NaN                     NaN
7              50.3400                    6.88
8               1.2500                    1.56
9               1.2400                    1.67
10              0.2300                    1.21
11              1.2500                    1.86
12              1.2413                     NaN
13              1.4289                     NaN
14              0.7300                    1.29
15              1.2500                    2.06
```

[29]:
```python
benchmark_results_df.to_parquet(
    FRAMEWORK_RESULTS,
    engine="pyarrow",
    compression="snappy",
    index=False
)

print(
    f"Ergebnisse gespeichert unter: "
    f"{FRAMEWORK_RESULTS}"
)
```

```
Ergebnisse gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\02_results.parquet
```

1.5.4 PySpark

Apache Spark ist ein Framework zur verteilten Verarbeitung großer Datenmengen. Mit PySpark kann Spark über Python verwendet werden.

Die Verarbeitung erfolgt grundsätzlich zunächst verzögert. Erst durch eine Aktion wie count() werden die Daten tatsächlich geladen und verarbeitet. Für den lokalen Test wird Spark mit allen verfügbaren CPU-Kernen gestartet.

Vor der Initialisierung werden zunächst Java sowie die temporären Spark-Verzeichnisse kontrolliert.

PySpark setzt dieselbe Berechnungsaufgabe mit Spark-DataFrame- beziehungsweise Spark-SQL-Operationen um.

Die SQL-Abfrage stellt dabei lediglich die frameworkgerechte technische Umsetzung der bereits

beschriebenen einheitlichen Berechnungslogik dar. Die Berechnungszeit misst die Ausführung dieser Abfrage.

Einrichtung und Initialisierung PySpark benötigt neben dem Python-Import eine zusätzliche Laufzeitumgebung. Dazu gehören insbesondere eine kompatible Java-Version, die gestartete Java Virtual Machine sowie eine aktive `SparkSession`.

Die Initialisierung umfasst unter anderem:

- Prüfung beziehungsweise Aktivierung von Java,
- Konfiguration der Umgebungsvariablen,
- Start der lokalen Spark-Anwendung,
- Aufbau der Verbindung zwischen Python und der JVM,
- Erstellung der `SparkSession`.

Die dafür benötigte Zeit wird einmalig beim Start des PySpark-Abschnitts gemessen. Sie wird nicht in die einzelnen Ergebnisse je Dateiformat einbezogen, da dieselbe `SparkSession` anschließend für mehrere Testfälle wiederverwendet wird.

Die Initialisierungszeit wird deshalb separat dokumentiert. Sie beschreibt den zusätzlichen technischen Einrichtungsaufwand von PySpark und ist nicht direkt mit der Ladezeit eines einzelnen Dateiformats gleichzusetzen.

[30]:
```
from pyspark.sql import SparkSession
```

[31]:
```
# 1. JAVA_HOME kontrollieren
java_path = os.environ.get("JAVA_HOME", "")

print(f"Genutztes JAVA_HOME: '{java_path}'")

if not java_path:
    print("Warnung: JAVA_HOME ist nicht gesetzt.")


# 2. Temporäre Spark-Pfade im Benutzerordner festlegen
user_home = os.path.expanduser("~")

spark_warehouse = os.path.join(
    user_home,
    "spark-warehouse"
)

spark_scratch = os.path.join(
    user_home,
    "spark-scratch"
)

print(f"Spark Warehouse wird angelegt in: {spark_warehouse}")
print(f"Spark Temp-Ordner wird angelegt in: {spark_scratch}")
```

```
# 3. Verzeichnisse sicher anlegen
os.makedirs(spark_warehouse, exist_ok=True)
os.makedirs(spark_scratch, exist_ok=True)

print("Spark-Verzeichnisse sind vorbereitet.")
```

```
Genutztes JAVA_HOME: 'C:\Users\alfa\.jdk\jdk-17.0.20.1+1'
Spark Warehouse wird angelegt in: C:\Users\alfa\spark-warehouse
Spark Temp-Ordner wird angelegt in: C:\Users\alfa\spark-scratch
Spark-Verzeichnisse sind vorbereitet.
```

[32]:
```
print("Initialisiere SparkSession ...")

start_spark_initialisierung = time.perf_counter()

spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("FrameworkEvaluation")
    .config("spark.sql.warehouse.dir", str(spark_warehouse))
    .config("spark.local.dir", str(spark_scratch))
    .config("spark.driver.host", "127.0.0.1")
    .config("spark.driver.bindAddress", "127.0.0.1")
    .config("spark.ui.port", "4050")
    .config(
        "spark.driver.extraJavaOptions",
        "-Djava.net.preferIPv4Stack=true "
        "-Dfile.encoding=UTF-8 "
        "-Dsun.jnu.encoding=UTF-8"
    )
    .config(
        "spark.executor.extraJavaOptions",
        "-Dfile.encoding=UTF-8 "
        "-Dsun.jnu.encoding=UTF-8"
    )
    .getOrCreate()
)

spark_initialisierungszeit = (
    time.perf_counter() - start_spark_initialisierung
)

print("SparkSession erfolgreich lokal mit UTF-8 gestartet.")
print(
    f"Spark-Initialisierung: "
```

```
    f"{spark_initialisierungszeit:.6f} s"
)
```

```
Initialisiere SparkSession …

C:\0_DA\Python312\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning:
PySpark does not yet fully support pandas >= 3.0.0. Some features may not work
correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()

SparkSession erfolgreich lokal mit UTF-8 gestartet.
Spark-Initialisierung: 16.712100 s
```

PySpark direkt Die vorbereiteten Dateien werden direkt mit PySpark eingelesen und anschließend über eine Spark-SQL-Abfrage verarbeitet.

[33]:
```
def measure_spark_direct(file_type, file_path, spark):
    """
    Misst die kombinierte End-to-End-Verarbeitung (Laden + SQL-Berechnung) mit␣
 ↪PySpark direkt von der Festplatte.
    """
    clean_memory()
    spark.catalog.clearCache()

    df = None
    tracemalloc.start()

    try:
        start_load = time.perf_counter()
        # Bei der Direkt-Methode kostet das reine Einlesen (Graph-Aufbau) fast␣
 ↪keine Zeit
        if file_type == "CSV":
            df = spark.read.option("header", True).option("inferSchema", True).
 ↪csv(file_path)
        elif file_type == "BZ2":
            df = spark.read.option("header", True).option("inferSchema", True).
 ↪option("compression", "bzip2").csv(file_path)
        elif file_type == "ZIP":
            df = spark.read.option("header", True).option("inferSchema", True).
 ↪option("compression", "zip").csv(file_path)
        elif file_type == "Parquet":
            df = spark.read.parquet(file_path)
        else:
            raise ValueError(f"Unbekanntes Dateiformat: {file_type}")

        load_time = time.perf_counter() - start_load

        _, peak_bytes = tracemalloc.get_traced_memory()
```

```
        ram_python_peak_mb = peak_bytes / (1024 * 1024)

        # Nadel für den isolierten Rechen-Peak nullen
        tracemalloc.reset_peak()

        # --- BERECHNUNGSPHASE (Deine SQL-Logik) ---
        start_calc = time.perf_counter()

        # Temporäre View registrieren
        df.createOrReplaceTempView("zug_daten")

        # SQL-Abfrage ([3, 4, 5] für category einsetzen)
        query = """
            SELECT
                state,
                COUNT(ID) as anzahl_zuege,
                AVG(CAST(departure_delay_m AS DOUBLE)) as␣
↪avg_abfahrts_verspaetung,
                MAX(CAST(departure_delay_m AS DOUBLE)) as max_verspaetung
            FROM zug_daten
            WHERE state IS NOT NULL
              AND category IN (3, 4, 5)
            GROUP BY state
            ORDER BY avg_abfahrts_verspaetung DESC
            LIMIT 5
        """

        # .collect() zwingt Spark, den optimierten Graphen von der Festplatte␣
↪weg auszuführen
        spark_result = spark.sql(query).collect()

        calc_time = time.perf_counter() - start_calc

        # Python-Skript Peaks auslesen
        _, peak_bytes_calc = tracemalloc.get_traced_memory()
        ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        # nur des Ergebnisobjektes
        size_bytes = sys.getsizeof(spark_result) + sum(sys.getsizeof(row) for␣
↪row in spark_result)
        ram_data_netto_mb = size_bytes / (1024 * 1024)

        row_count = df.count()
        result_text = f"{len(df.columns)} Spalten, {row_count} Zeilen, {df.rdd.
↪getNumPartitions()} Partitionen"
```

```
        row = {
            "Framework": "PySpark",
            "Methode": "direkt",
            "Format": file_type,
            "Kompatibilität": "geeignet",
            "Ladezeit (s)": round(load_time, 6),
            "Berechnungszeit (s)": round(calc_time, 6),
            "Zeilen": row_count,
            "Spalten": len(df.columns),
            "Status": "erfolgreich",
            "Ergebnis": result_text,
            "Fehler": None,
            "RAM_Data_Netto_MB": round(ram_data_netto_mb, 4),
            "RAM_Python_Peak_MB": round(ram_python_peak_mb, 4),
            "RAM_Python_Calc_Peak_MB": round(ram_python_calc_peak_mb, 2)
        }

    except Exception as error:
        row = {
            "Framework": "PySpark",
            "Methode": "direkt",
            "Format": file_type,
            "Kompatibilität": "Fehler",
            "Ladezeit (s)": None,
            "Berechnungszeit (s)": None,
            "Zeilen": None,
            "Spalten": None,
            "Status": "fehlgeschlagen",
            "Ergebnis": "Direkte SQL-Berechnung in PySpark fehlgeschlagen",
            "Fehler": repr(error),
            "RAM_Data_Netto_MB": None,
            "RAM_Python_Peak_MB": None,
            "RAM_Python_Calc_Peak_MB": None
        }

    finally:
        tracemalloc.stop()
        try:
            spark.catalog.dropTempView("zug_daten")
        except Exception:
            pass
        if df is not None:
            del df
        clean_memory()

    return row
```

[34]:
```
spark_direct_results = []
for format_name, dateipfad in paths_to_check_relative_files.items():
    print(f"Teste PySpark direct mit {format_name} ...")

    result = measure_spark_direct(
        spark=spark,
        file_type=format_name,
        file_path=paths_to_check_relative_files[format_name]
    )
    spark_direct_results.append(result)

spark_direct_df = pd.DataFrame(spark_direct_results)

display(spark_direct_df)
```

```
Teste PySpark direct mit ZIP …
Teste PySpark direct mit BZ2 …
Teste PySpark direct mit Parquet …
Teste PySpark direct mit CSV …
```

	Framework	Methode	Format	Kompatibilität	Ladezeit (s)	Berechnungszeit (s)	Zeilen	Spalten	Status	Ergebnis	Fehler	RAM_Data_Netto_MB	RAM_Python_Peak_MB	RAM_Python_Calc_Peak_MB
0	PySpark	direkt	ZIP	Fehler	NaN	NaN	NaN	NaN	fehlgeschlagen	Direkte SQL-Berechnung in PySpark fehlgeschlagen	IllegalArgumentException()	NaN	NaN	NaN
1	PySpark	direkt	BZ2	geeignet	11.610254	10.098519	2061357.0	20.0	erfolgreich	20 Spalten, 2061357 Zeilen, 4 Partitionen	NaN	0.0005	0.3870	0.67
2	PySpark	direkt	Parquet	geeignet	0.545169	1.376866	2061357.0	20.0	erfolgreich	20 Spalten, 2061357 Zeilen, 4 Partitionen	NaN	0.0005	0.0115	0.15
3	PySpark	direkt	CSV	geeignet	3.827554	3.026352	2061357.0	20.0	erfolgreich	20 Spalten, 2061357 Zeilen, 6 Partitionen	NaN	0.0005	0.0111	

```
3                    0.15
```

[35]:
```
benchmark_results.extend(spark_direct_results)

benchmark_results_df = pd.DataFrame(benchmark_results)

display(benchmark_results_df)
```

```
   Framework   Methode   Format   Kompatibilität  Ladezeit (s)  \
0     Pandas    direkt      ZIP         geeignet     23.035550
1     Pandas    direkt      BZ2         geeignet     36.412793
2     Pandas    direkt  Parquet         geeignet      0.515125
3     Pandas    direkt      CSV         geeignet     20.954767
4     Pandas  Chunking      ZIP         geeignet     23.596085
5     Pandas  Chunking      BZ2         geeignet     37.948140
6     Pandas  Chunking  Parquet  nicht anwendbar           NaN
7     Pandas  Chunking      CSV         geeignet     20.912869
8       Dask    direkt      ZIP           Fehler      0.066706
9       Dask    direkt      BZ2           Fehler      0.052605
10      Dask    direkt  Parquet         geeignet      0.047000
11      Dask    direkt      CSV         geeignet      0.055500
12      Dask    cached      ZIP           Fehler     56.807416
13      Dask    cached      BZ2           Fehler    118.478661
14      Dask    cached  Parquet         geeignet      2.401278
15      Dask    cached      CSV         geeignet      4.926066
16   PySpark    direkt      ZIP           Fehler           NaN
17   PySpark    direkt      BZ2         geeignet     11.610254
18   PySpark    direkt  Parquet         geeignet      0.545169
19   PySpark    direkt      CSV         geeignet      3.827554

    Berechnungszeit (s)     Zeilen  Spalten          Status  \
0              0.795007  2061357.0     20.0     erfolgreich
1              0.745345  2061357.0     20.0     erfolgreich
2              0.868837  2061357.0     20.0     erfolgreich
3              0.647345  2061357.0     20.0     erfolgreich
4              0.011319  2061357.0     20.0     erfolgreich
5              0.010420  2061357.0     20.0     erfolgreich
6                   NaN        NaN      NaN    übersprungen
7              0.011362  2061357.0     20.0     erfolgreich
8             53.702210        NaN      NaN  fehlgeschlagen
9            115.299709        NaN      NaN  fehlgeschlagen
10             1.532400  2061357.0     20.0     erfolgreich
11             6.257700  2061357.0     20.0     erfolgreich
12                  NaN        NaN      NaN  fehlgeschlagen
13                  NaN        NaN      NaN  fehlgeschlagen
14             4.676405  2061357.0     20.0     erfolgreich
15             0.413779  2061357.0     20.0     erfolgreich
16                  NaN        NaN      NaN  fehlgeschlagen
```

```
17            10.098519  2061357.0     20.0     erfolgreich
18             1.376866  2061357.0     20.0     erfolgreich
19             3.026352  2061357.0     20.0     erfolgreich

                                          Ergebnis  \
0                       20 Spalten, 2061357 Zeilen
1                       20 Spalten, 2061357 Zeilen
2                       20 Spalten, 2061357 Zeilen
3                       20 Spalten, 2061357 Zeilen
4              20 Spalten, 2061357 Zeilen, 21 Chunks
5              20 Spalten, 2061357 Zeilen, 21 Chunks
6     Pandas read_parquet unterstützt kein chunksize
7              20 Spalten, 2061357 Zeilen, 21 Chunks
8           Direkte Berechnung in Dask fehlgeschlagen
9           Direkte Berechnung in Dask fehlgeschlagen
10          20 Spalten, 2061357 Zeilen, 1 Partitionen
11         20 Spalten, 2061357 Zeilen, 12 Partitionen
12           Cached Berechnung in Dask fehlgeschlagen
13           Cached Berechnung in Dask fehlgeschlagen
14          20 Spalten, 2061357 Zeilen, 1 Partitionen
15         20 Spalten, 2061357 Zeilen, 12 Partitionen
16  Direkte SQL-Berechnung in PySpark fehlgeschlagen
17          20 Spalten, 2061357 Zeilen, 4 Partitionen
18          20 Spalten, 2061357 Zeilen, 4 Partitionen
19          20 Spalten, 2061357 Zeilen, 6 Partitionen

                                              Fehler  RAM_Data_Netto_MB  \
0                                                NaN           947.3900
1                                                NaN           947.3900
2                                                NaN           947.3900
3                                                NaN           947.3900
4                                                NaN             0.0003
5                                                NaN             0.0003
6                                                NaN                NaN
7                                                NaN             0.0003
8   KilledWorker(('read-csv-ad7fe2a87dab5d3bbdfff3…                NaN
9   KilledWorker(('read-csv-7166cdbdd63db72df82611…                NaN
10                                               NaN             0.0003
11                                               NaN             0.0003
12  KilledWorker(('read-_to_string_dtype-5dec73148…                NaN
13  KilledWorker(('read-_to_string_dtype-ceb7586fa…                NaN
14                                               NaN           947.3900
15                                               NaN           947.3900
16                       IllegalArgumentException()                NaN
17                                               NaN             0.0005
18                                               NaN             0.0005
19                                               NaN             0.0005
```

	RAM_Python_Peak_MB	RAM_Python_Calc_Peak_MB
0	777.5500	251.61
1	777.5500	251.56
2	127.7400	141.99
3	777.5300	251.56
4	50.6100	6.88
5	49.9100	6.87
6	NaN	NaN
7	50.3400	6.88
8	1.2500	1.56
9	1.2400	1.67
10	0.2300	1.21
11	1.2500	1.86
12	1.2413	NaN
13	1.4289	NaN
14	0.7300	1.29
15	1.2500	2.06
16	NaN	NaN
17	0.3870	0.67
18	0.0115	0.15
19	0.0111	0.15

[36]:
```
benchmark_results_df.to_parquet(
    FRAMEWORK_RESULTS,
    engine="pyarrow",
    compression="snappy",
    index=False
)

print(
    f"Ergebnisse gespeichert unter: "
    f"{FRAMEWORK_RESULTS}"
)
```

```
Ergebnisse gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\02_results.parquet
```

PySpark cached Bei dieser Variante wird das eingelesene Spark-DataFrame zunächst persistiert und anschließend materialisiert. Dadurch stehen die Daten für die nachfolgende Berechnung im Spark-Speicher beziehungsweise – abhängig vom Speicherbedarf – zusätzlich auf der Festplatte zur Verfügung.

Die Messung umfasst daher:

- Aufbau des Spark-DataFrames,
- Materialisierung des Caches,
- Ausführung der einheitlichen Berechnungsaufgabe,
- verwendeten Speicher beziehungsweise Datenumfang,
- Lade- und Berechnungszeit.

Das Caching kann wiederholte Berechnungen auf denselben Daten beschleunigen. Es verursacht jedoch zunächst zusätzlichen Zeit- und Speicheraufwand. Deshalb wird diese Variante nicht als grundsätzlich schneller bewertet, sondern als andere Verarbeitungsstrategie eingeordnet.

Nach Abschluss des Testfalls wird der Cache mit `unpersist()` wieder freigegeben.

> Hinweis: Mit `persist(StorageLevel.MEMORY_AND_DISK)` könnten auch größere Dateien mit PySpark behandelt werden.
>
> Spark verwendet dabei zunächst den Arbeitsspeicher und lagert Daten bei nicht ausreichendem Speicherbedarf auf die lokale Festplatte aus.

[37]:
```python
def measure_spark_cached(file_type, file_path, spark):
    """
    Misst das Laden von PySpark-Daten in den JVM-RAM (cached) und führt danach
    eine isolierte In-Memory-SQL-Berechnung durch.
    """
    clean_memory()
    spark.catalog.clearCache()

    df = None
    tracemalloc.start()

    try:
        # --- LADEPHASE (In den RAM der JVM/Worker laden via .persist()) ---
        start_load = time.perf_counter()

        if file_type == "CSV":
            df = spark.read.option("header", True).option("inferSchema", True).
↪csv(file_path)
        elif file_type == "BZ2":
            df = spark.read.option("header", True).option("inferSchema", True).
↪option("compression", "bzip2").csv(file_path)
        elif file_type == "ZIP":
            df = spark.read.option("header", True).option("inferSchema", True).
↪option("compression", "zip").csv(file_path)
        elif file_type == "Parquet":
            df = spark.read.parquet(file_path)
        else:
            raise ValueError(f"Unbekanntes Dateiformat: {file_type}")

        # Daten in der JVM cachen
        df = df.persist()

        # Eine Aktion erzwingen, damit Spark die Daten physisch in den RAM lädt
        row_count = df.count()
        load_time = time.perf_counter() - start_load

        # Framework-interne Datengröße aus der JVM auslesen
```

```
        try:
            sc = spark.sparkContext
            java_estimator = sc._jvm.org.apache.spark.util.SizeEstimator
            ram_data_netto_mb = java_estimator.estimate(df._jdf.rdd()) / (1024␣
↪* 1024)
        except Exception:
            ram_data_netto_mb = 0.0

        # Python Lade-Peak auslesen (wird nahe 0 sein)
        _, peak_bytes_load = tracemalloc.get_traced_memory()
        ram_python_peak_mb = peak_bytes_load / (1024 * 1024)

        # Nadel für die anschließende Berechnung nullen
        tracemalloc.reset_peak()

        # --- 2. BERECHNUNGSPHASE (Reine In-Memory SQL Transformation) ---
        start_calc = time.perf_counter()

        # Temporäre View auf das bereits gecachte DataFrame registrieren
        df.createOrReplaceTempView("zug_daten")

        query = """
            SELECT
                state,
                COUNT(ID) as anzahl_zuege,
                AVG(CAST(departure_delay_m AS DOUBLE)) as␣
↪avg_abfahrts_verspaetung,
                MAX(CAST(departure_delay_m AS DOUBLE)) as max_verspaetung
            FROM zug_daten
            WHERE state IS NOT NULL
              AND category IN (3, 4, 5)
            GROUP BY state
            ORDER BY avg_abfahrts_verspaetung DESC
            LIMIT 5
        """

        # Abfrage auf In-Memory-Daten ausführen
        spark_result = spark.sql(query).collect()

        calc_time = time.perf_counter() - start_calc

        # Berechnungs-Peak des Hauptprozesses erfassen
        _, peak_bytes_calc = tracemalloc.get_traced_memory()
        ram_python_calc_peak_mb = peak_bytes_calc / (1024 * 1024)

        result_text = f"{len(df.columns)} Spalten, {df.rdd.getNumPartitions()}␣
↪Partitionen"
```

```
        row = {
            "Framework": "PySpark",
            "Methode": "cached",
            "Format": file_type,
            "Kompatibilität": "geeignet",
            "Ladezeit (s)": round(load_time, 6),
            "Berechnungszeit (s)": round(calc_time, 6),
            "Zeilen": row_count,
            "Spalten": len(df.columns),
            "Status": "erfolgreich",
            "Ergebnis": result_text,
            "Fehler": None,
            "RAM_Data_Netto_MB": round(ram_data_netto_mb, 2),
            "RAM_Python_Peak_MB": round(ram_python_peak_mb, 2),
            "RAM_Python_Calc_Peak_MB": round(ram_python_calc_peak_mb, 2)
        }

    except Exception as error:
        row = {
            "Framework": "PySpark",
            "Methode": "cached",
            "Format": file_type,
            "Kompatibilität": "Fehler",
            "Ladezeit (s)": None,
            "Berechnungszeit (s)": None,
            "Zeilen": None,
            "Spalten": None,
            "Status": "fehlgeschlagen",
            "Ergebnis": "Cached SQL-Berechnung in PySpark fehlgeschlagen",
            "Fehler": repr(error),
            "RAM_Data_Netto_MB": None,
            "RAM_Python_Peak_MB": None,
            "RAM_Python_Calc_Peak_MB": None
        }

    finally:
        tracemalloc.stop()
        try:
            spark.catalog.dropTempView("zug_daten")
        except Exception:
            pass
        if df is not None:
            df.unpersist()
            del df
        clean_memory()
```

```
    return row
```

[38]:
```
spark_cached_results = []
for format_name, dateipfad in paths_to_check_relative_files.items():
    print(f"Teste PySpark cached mit {format_name} ...")

    result = measure_spark_cached(
        spark=spark,
        file_type=format_name,
        file_path=paths_to_check_relative_files[format_name]
    )
    spark_cached_results.append(result)

spark_cached_df = pd.DataFrame(spark_cached_results)

display(spark_cached_df)
```

```
Teste PySpark cached mit ZIP …
Teste PySpark cached mit BZ2 …
Teste PySpark cached mit Parquet …
Teste PySpark cached mit CSV …
```

	Framework	Methode	Format	Kompatibilität	Ladezeit (s)	Berechnungszeit (s)	Zeilen	Spalten	Status	Ergebnis	Fehler	RAM_Data_Netto_MB	RAM_Python_Peak_MB	RAM_Python_Calc_Peak_MB
0	PySpark	cached	ZIP	Fehler	NaN	NaN	NaN	NaN	fehlgeschlagen	Cached SQL-Berechnung in PySpark fehlgeschlagen	IllegalArgumentException()	NaN	NaN	NaN
1	PySpark	cached	BZ2	geeignet	21.406243	0.542081	2061357.0	20.0	erfolgreich	20 Spalten, 4 Partitionen	NaN	343.81	0.01	NaN
2	PySpark	cached	Parquet	geeignet	6.825002	0.507786	2061357.0	20.0	erfolgreich	20 Spalten, 4 Partitionen	NaN	348.58	0.01	NaN
3	PySpark	cached	CSV	geeignet	12.031365	0.381001	2061357.0	20.0	erfolgreich	20 Spalten, 6 Partitionen	NaN	343.97	0.01	NaN

```
1                    0.16
2                    0.15
3                    0.15
```

[39]:
```
benchmark_results.extend(spark_cached_results)

benchmark_results_df = pd.DataFrame(benchmark_results)

display(benchmark_results_df)
```

```
   Framework   Methode   Format   Kompatibilität  Ladezeit (s)  \
0     Pandas    direkt      ZIP         geeignet     23.035550
1     Pandas    direkt      BZ2         geeignet     36.412793
2     Pandas    direkt  Parquet         geeignet      0.515125
3     Pandas    direkt      CSV         geeignet     20.954767
4     Pandas  Chunking      ZIP         geeignet     23.596085
5     Pandas  Chunking      BZ2         geeignet     37.948140
6     Pandas  Chunking  Parquet  nicht anwendbar           NaN
7     Pandas  Chunking      CSV         geeignet     20.912869
8       Dask    direkt      ZIP           Fehler      0.066706
9       Dask    direkt      BZ2           Fehler      0.052605
10      Dask    direkt  Parquet         geeignet      0.047000
11      Dask    direkt      CSV         geeignet      0.055500
12      Dask    cached      ZIP           Fehler     56.807416
13      Dask    cached      BZ2           Fehler    118.478661
14      Dask    cached  Parquet         geeignet      2.401278
15      Dask    cached      CSV         geeignet      4.926066
16   PySpark    direkt      ZIP           Fehler           NaN
17   PySpark    direkt      BZ2         geeignet     11.610254
18   PySpark    direkt  Parquet         geeignet      0.545169
19   PySpark    direkt      CSV         geeignet      3.827554
20   PySpark    cached      ZIP           Fehler           NaN
21   PySpark    cached      BZ2         geeignet     21.406243
22   PySpark    cached  Parquet         geeignet      6.825002
23   PySpark    cached      CSV         geeignet     12.031365

    Berechnungszeit (s)     Zeilen  Spalten          Status  \
0              0.795007  2061357.0     20.0     erfolgreich
1              0.745345  2061357.0     20.0     erfolgreich
2              0.868837  2061357.0     20.0     erfolgreich
3              0.647345  2061357.0     20.0     erfolgreich
4              0.011319  2061357.0     20.0     erfolgreich
5              0.010420  2061357.0     20.0     erfolgreich
6                   NaN        NaN      NaN    übersprungen
7              0.011362  2061357.0     20.0     erfolgreich
8             53.702210        NaN      NaN  fehlgeschlagen
9            115.299709        NaN      NaN  fehlgeschlagen
10             1.532400  2061357.0     20.0     erfolgreich
```

11	6.257700	2061357.0	20.0	erfolgreich
12	NaN	NaN	NaN	fehlgeschlagen
13	NaN	NaN	NaN	fehlgeschlagen
14	4.676405	2061357.0	20.0	erfolgreich
15	0.413779	2061357.0	20.0	erfolgreich
16	NaN	NaN	NaN	fehlgeschlagen
17	10.098519	2061357.0	20.0	erfolgreich
18	1.376866	2061357.0	20.0	erfolgreich
19	3.026352	2061357.0	20.0	erfolgreich
20	NaN	NaN	NaN	fehlgeschlagen
21	0.542081	2061357.0	20.0	erfolgreich
22	0.507786	2061357.0	20.0	erfolgreich
23	0.381001	2061357.0	20.0	erfolgreich

	Ergebnis
0	20 Spalten, 2061357 Zeilen
1	20 Spalten, 2061357 Zeilen
2	20 Spalten, 2061357 Zeilen
3	20 Spalten, 2061357 Zeilen
4	20 Spalten, 2061357 Zeilen, 21 Chunks
5	20 Spalten, 2061357 Zeilen, 21 Chunks
6	Pandas read_parquet unterstützt kein chunksize
7	20 Spalten, 2061357 Zeilen, 21 Chunks
8	Direkte Berechnung in Dask fehlgeschlagen
9	Direkte Berechnung in Dask fehlgeschlagen
10	20 Spalten, 2061357 Zeilen, 1 Partitionen
11	20 Spalten, 2061357 Zeilen, 12 Partitionen
12	Cached Berechnung in Dask fehlgeschlagen
13	Cached Berechnung in Dask fehlgeschlagen
14	20 Spalten, 2061357 Zeilen, 1 Partitionen
15	20 Spalten, 2061357 Zeilen, 12 Partitionen
16	Direkte SQL-Berechnung in PySpark fehlgeschlagen
17	20 Spalten, 2061357 Zeilen, 4 Partitionen
18	20 Spalten, 2061357 Zeilen, 4 Partitionen
19	20 Spalten, 2061357 Zeilen, 6 Partitionen
20	Cached SQL-Berechnung in PySpark fehlgeschlagen
21	20 Spalten, 4 Partitionen
22	20 Spalten, 4 Partitionen
23	20 Spalten, 6 Partitionen

	Fehler	RAM_Data_Netto_MB
0	NaN	947.3900
1	NaN	947.3900
2	NaN	947.3900
3	NaN	947.3900
4	NaN	0.0003
5	NaN	0.0003
6	NaN	NaN

7		NaN	0.0003
8	KilledWorker(('read-csv-ad7fe2a87dab5d3bbdfff3…		NaN
9	KilledWorker(('read-csv-7166cdbdd63db72df82611…		NaN
10		NaN	0.0003
11		NaN	0.0003
12	KilledWorker(('read-_to_string_dtype-5dec73148…		NaN
13	KilledWorker(('read-_to_string_dtype-ceb7586fa…		NaN
14		NaN	947.3900
15		NaN	947.3900
16	IllegalArgumentException()		NaN
17		NaN	0.0005
18		NaN	0.0005
19		NaN	0.0005
20	IllegalArgumentException()		NaN
21		NaN	343.8100
22		NaN	348.5800
23		NaN	343.9700

	RAM_Python_Peak_MB	RAM_Python_Calc_Peak_MB
0	777.5500	251.61
1	777.5500	251.56
2	127.7400	141.99
3	777.5300	251.56
4	50.6100	6.88
5	49.9100	6.87
6	NaN	NaN
7	50.3400	6.88
8	1.2500	1.56
9	1.2400	1.67
10	0.2300	1.21
11	1.2500	1.86
12	1.2413	NaN
13	1.4289	NaN
14	0.7300	1.29
15	1.2500	2.06
16	NaN	NaN
17	0.3870	0.67
18	0.0115	0.15
19	0.0111	0.15
20	NaN	NaN
21	0.0100	0.16
22	0.0100	0.15
23	0.0100	0.15

[40]:
```
benchmark_results_df.to_parquet(
    FRAMEWORK_RESULTS,
    engine="pyarrow",
```

```
    compression="snappy",
    index=False
)

print(
    f"Ergebnisse gespeichert unter: "
    f"{FRAMEWORK_RESULTS}"
)
```

```
Ergebnisse gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\02_results.parquet
```

1.6 Visualisierung und Ergebnisvergleich

Nach Abschluss der Messungen werden die Ergebnisse zunächst hinsichtlich ihrer grundsätzlichen Eignung visualisiert. Dazu wird eine Heatmap erstellt, in der die erfolgreichen Kombinationen aus Framework, Methode und Dateiformat übersichtlich dargestellt werden.

Die Heatmap dient als erster Überblick über die Verfügbarkeit und Eignung der getesteten Kombinationen. Fehlgeschlagene oder nicht unterstützte Verfahren werden dabei kenntlich gemacht.

Anschließend folgen detaillierte Vergleiche der erfolgreichen Testfälle. Dabei werden insbesondere folgende Kennzahlen gegenübergestellt:

- Ladezeit,
- Berechnungszeit,
- Gesamte Verarbeitungszeit (nur eine kleine Abfrage bei Berechnung),
- Python-Speicher-Peak,
- Berechnungs-Speicher-Peak,
- gemessener Daten- beziehungsweise Framework-Speicher,
- Zeilen- und Spaltenanzahl.

Die Visualisierungen bilden ausschließlich die tatsächlich gespeicherten Messwerte ab. Die fachliche Berechnung ist bei allen Frameworks gleich; die Unterschiede ergeben sich aus der jeweiligen technischen Umsetzung.

[41]:
```
df_framework_results = pd.read_parquet(FRAMEWORK_RESULTS)
```

[42]:
```
import plotly.express as px

# Daten für die Heatmap vorbereiten
# Framework und Methode kombiniert
heatmap_df = df_framework_results.copy()
heatmap_df["Verarbeitungsmodus"] = heatmap_df["Framework"] + " (" +␣
 ↪heatmap_df["Methode"] + ")"

# Status in Zahlen für die Farb-Skala übersetzen
# 2= Erfolgreich, 1 = Übersprungen, 0 = Fehlgeschlagen
status_mapping = {"erfolgreich": 2, "übersprungen": 1, "fehlgeschlagen": 0}
heatmap_df["Status_Wert"] = heatmap_df["Status"].map(status_mapping)
```

```
# Pivot-Tabelle erstellen:
pivot_df = heatmap_df.pivot(index="Verarbeitungsmodus", columns="Format",␣
 ↪values="Status_Wert")

# Text-Labels in der Heatmap anzeigen (Text-Matrix)
text_df = heatmap_df.pivot(index="Verarbeitungsmodus", columns="Format",␣
 ↪values="Status")
# Text verschönern für die Grafik
text_df = text_df.replace({"erfolgreich": "OK", "übersprungen": "N/A",␣
 ↪"fehlgeschlagen": "Fehler"})

fig = px.imshow(
    pivot_df,
    labels=dict(x="Dateiformat", y="Framework & Methode", color="Robustheit"),
    x=pivot_df.columns,
    y=pivot_df.index,
    # Farbpalette: Rot (0), Gelb (1), Grün (2)
    color_continuous_scale=[[0.0, "#EF553B"], [0.5, "#FECB52"], [1.0,␣
 ↪"#00CC96"]],
    text_auto=False,
    title="Kompatibilitäts- und Robustheitsmatrix der Frameworks",
    height=900
)

# Eigene Text-Labels
fig.update_traces(
    text=text_df.values,
    texttemplate="%{text}",
    textfont=dict(size=14, color="white")
)

# Layout optimieren (Farbleiste ausblenden, da Text eindeutig ist)
fig.update_layout(
    coloraxis_showscale=False,
    title_x=0.5, # Titel zentrieren
    xaxis_nticks=4,
    template="plotly_white"
)

# Grafik im Notebook anzeigen
fig.show()
```



[43]:
```
# Daten für die grafische Darstellung aufbereiten
chart_df = heatmap_df.copy()


# Fehlgeschlagene / Übersprungene Werte für die Zeitgrafik filtern, um␣
 ↪Verzerrungen zu vermeiden
success_df = chart_df[chart_df["Status"] == "erfolgreich"].copy()
success_df["Gesamtzeit (s)"] = success_df["Ladezeit (s)"] +␣
 ↪success_df["Berechnungszeit (s)"]

fig_time_load = px.bar(
    success_df,
    x="Format",
    y="Ladezeit (s)",
    color="Verarbeitungsmodus",
    barmode="group",
    title="Ladezeiten im Vergleich",
    labels={"Gesamtzeit (s)": "Zeit in Sekunden", "Format": "Dateiformat"},
    text_auto='.2f',
    height=900,
)
fig_time_load.update_layout(title_x=0.5, template="plotly_white")
fig_time_load.show()


fig_time_calc = px.bar(
    success_df,
    x="Format",
    y="Berechnungszeit (s)",
```

```
    color="Verarbeitungsmodus",
    barmode="group",
    title="Berechnungszeiten einer kleiner Abfrage im Vergleich",
    labels={"Gesamtzeit (s)": "Zeit in Sekunden", "Format": "Dateiformat"},
    text_auto='.2f',
    height=900,
)
fig_time_calc.update_layout(title_x=0.5, template="plotly_white")
fig_time_calc.show()

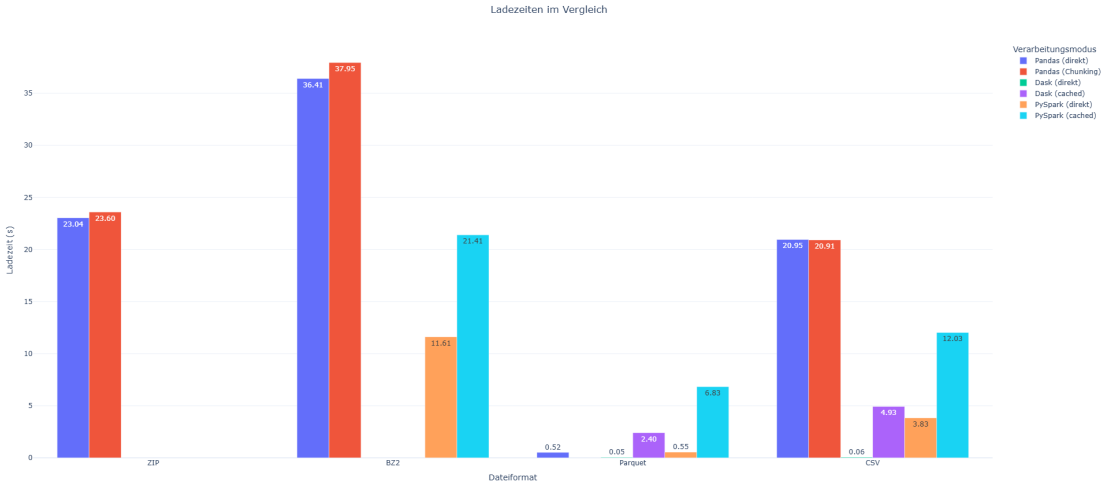
# aber hier nicht so representativ, wegen nur einer Abfrage
fig_time = px.bar(
    success_df,
    x="Format",
    y="Gesamtzeit (s)",
    color="Verarbeitungsmodus",
    barmode="group",
    title="Gesamtlaufzeit (Laden + Berechnung) im Vergleich",
    labels={"Gesamtzeit (s)": "Zeit in Sekunden", "Format": "Dateiformat"},
    text_auto='.2f',
    height=900,
)
fig_time.update_layout(title_x=0.5, template="plotly_white")
fig_time.show()

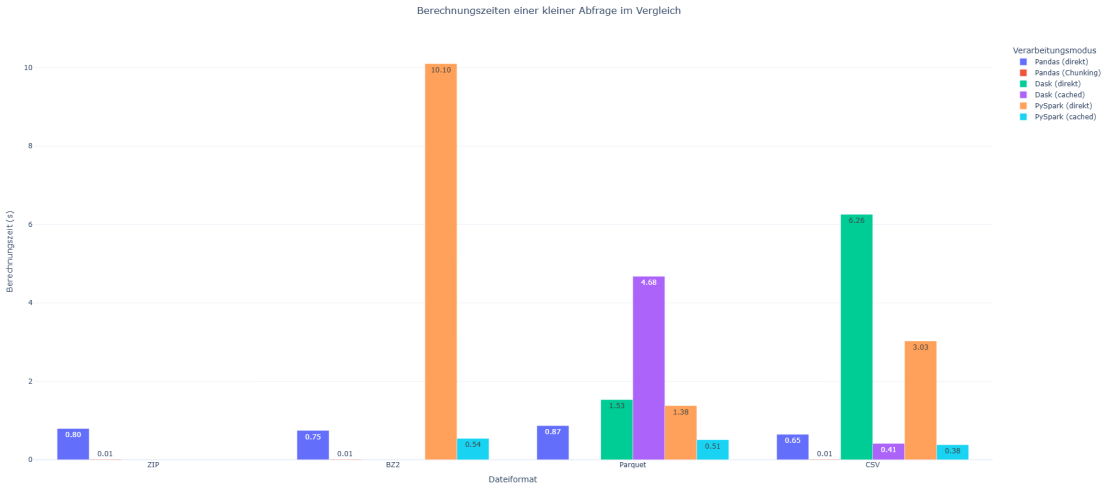
fig_ram_data = px.bar(
    success_df,
    x="Verarbeitungsmodus",
    y="RAM_Data_Netto_MB",
    color="Format",
    barmode="group",
    title="Arbeitsspeicher für Daten",
    labels={"RAM_Python_Calc_Peak_MB": "RAM Peak (MB)", "Verarbeitungsmodus":␣
 ↪"Framework & Methode"},
    text_auto='.1f',
    height=900,
)
fig_ram_data.update_layout(title_x=0.5, template="plotly_white")
fig_ram_data.show()

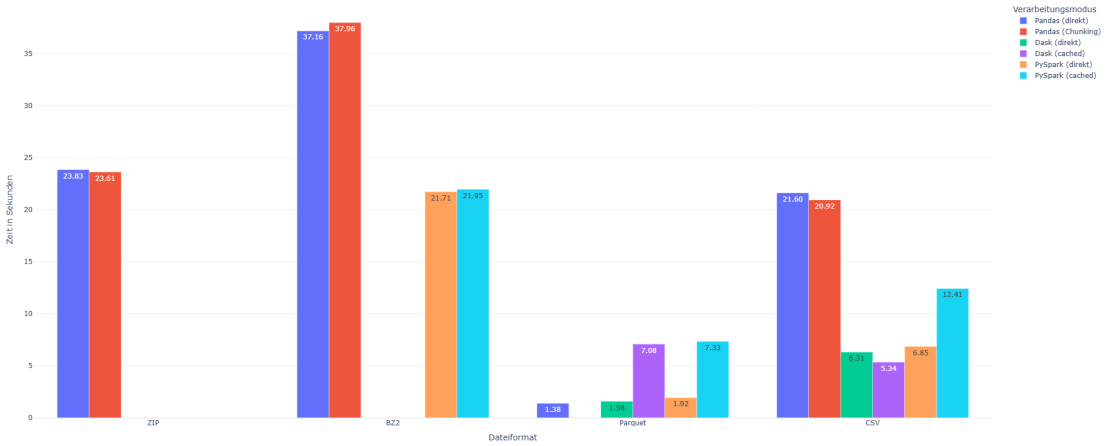
fig_ram_load = px.bar(
    success_df,
    x="Verarbeitungsmodus",
    y="RAM_Python_Peak_MB",
    color="Format",
    barmode="group",
    title="Isolierter Python-Arbeitsspeicher-Peak während des Ladens",
```

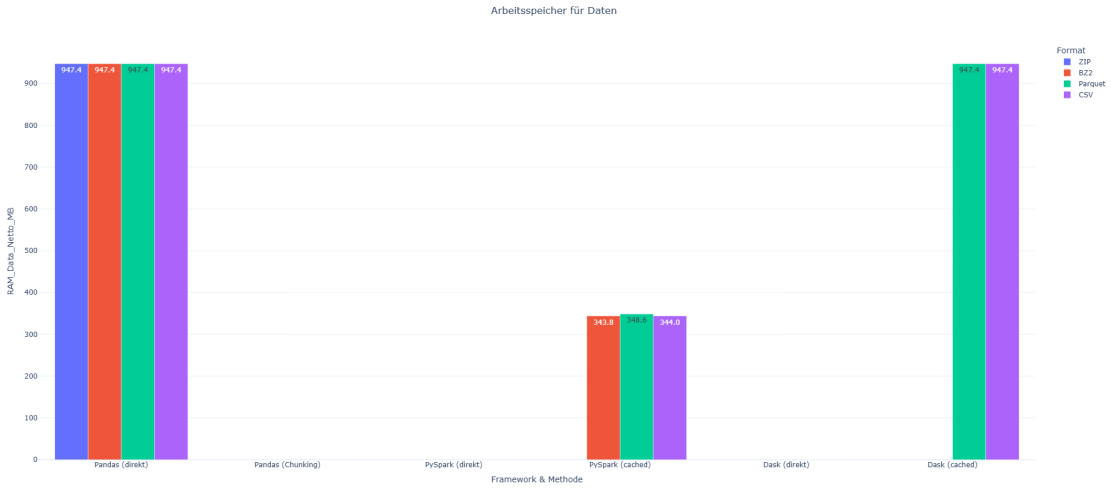
```
    labels={"RAM_Python_Calc_Peak_MB": "RAM Peak (MB)", "Verarbeitungsmodus":␣
 ↪"Framework & Methode"},
    text_auto='.1f',
    height=900,
)
fig_ram_load.update_layout(title_x=0.5, template="plotly_white")
fig_ram_load.show()

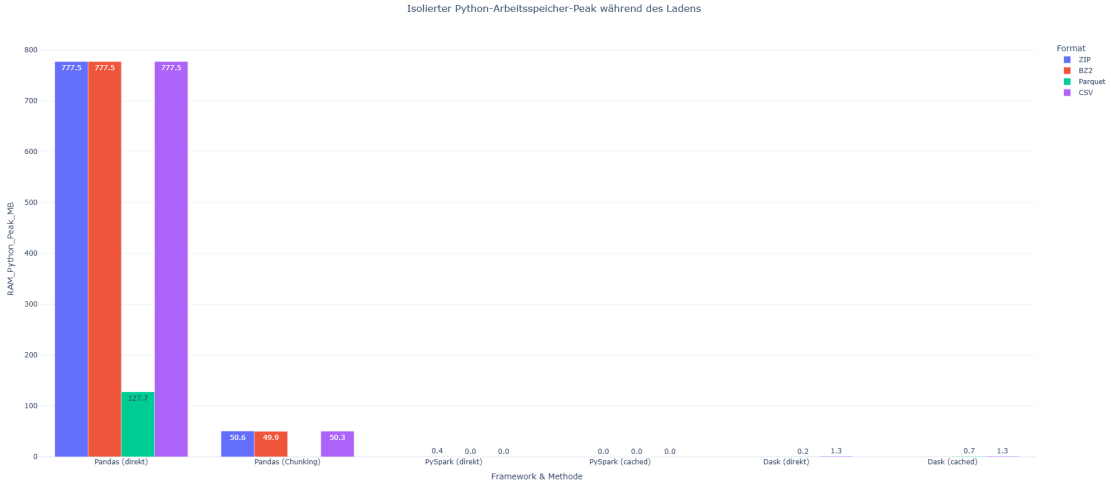
fig_ram_calc = px.bar(
    success_df,
    x="Verarbeitungsmodus",
    y="RAM_Python_Calc_Peak_MB",
    color="Format",
    barmode="group",
    title="Isolierter Python-Arbeitsspeicher-Peak während der Berechnung",
    labels={"RAM_Python_Calc_Peak_MB": "RAM Peak (MB)", "Verarbeitungsmodus":␣
 ↪"Framework & Methode"},
    text_auto='.1f',
    height=900,
)
fig_ram_calc.update_layout(title_x=0.5, template="plotly_white")
fig_ram_calc.show()
```

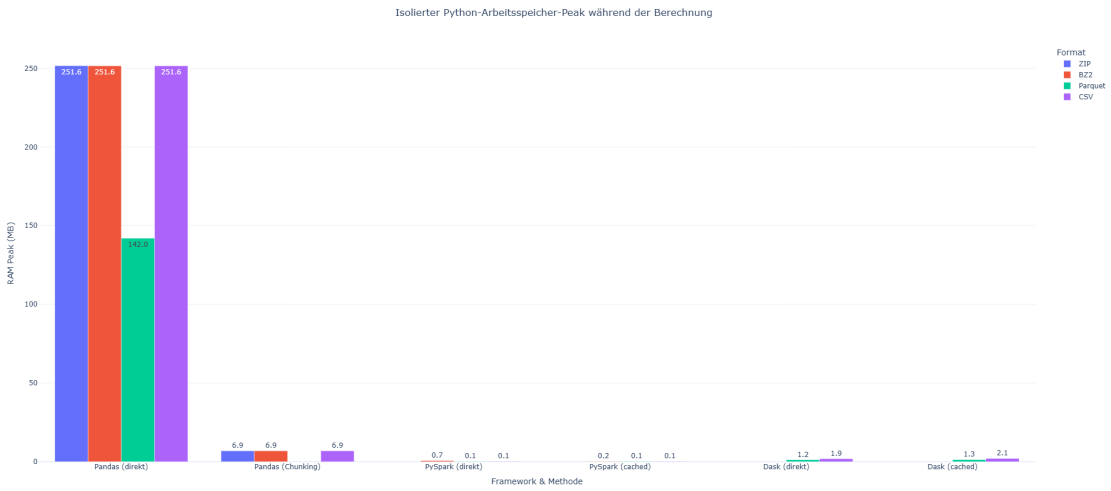



Berechnungszeiten einer kleiner Abfrage im Vergleich
Verarbeitungsmodus
Pandas (direkt)
Pandas (Chunking)
Dask (direkt)
Dask (cached)
PySpark (direkt)
PySpark (cached)
Berechnungszeit (s)
0
2
4
6
8
10
0.80
0.01
0.75
0.01
10.10
0.54
0.87
1.53
4.68
1.38
0.51
0.65
0.01
6.26
0.41
3.03
0.38
ZIP
BZ2
Parquet
CSV
Dateiformat


Gesamtlaufzeit (Laden + Berechnung) im Vergleich
Verarbeitungsmodus
Pandas (direkt)
Pandas (Chunking)
Dask (direkt)
Dask (cached)
PySpark (direkt)
PySpark (cached)
Zeit in Sekunden
0
5
10
15
20
25
30
35
23.83
23.61
37.16
37.96
21.71
21.95
1.38
1.58
7.08
1.92
7.33
21.60
20.92
6.31
5.34
6.85
12.41
ZIP
BZ2
Parquet
CSV
Dateiformat


Arbeitsspeicher für Daten
RAM_Data_Netto_MB
Framework & Methode
Format
ZIP
BZ2
Parquet
CSV
947.4
947.4
947.4
947.4
343.8
348.6
344.0
947.4
947.4
Pandas (direkt)
Pandas (Chunking)
PySpark (direkt)
PySpark (cached)
Dask (direkt)
Dask (cached)


Isolierter Python-Arbeitsspeicher-Peak während des Ladens
RAM_Python_Peak_MB
Framework & Methode
Format
ZIP
BZ2
Parquet
CSV
777.5
777.5
127.7
777.5
50.6
49.9
50.3
0.4
0.0
0.0
0.0
0.0
0.0
0.2
1.3
0.7
1.3
Pandas (direkt)
Pandas (Chunking)
PySpark (direkt)
PySpark (cached)
Dask (direkt)
Dask (cached)



2 Schlussfolgerungen und Empfehlung für das Projekt

2.1 Einordnung der Untersuchung

Die durchgeführten Tests vergleichen Pandas, Dask und PySpark anhand derselben fachlichen Berechnungsaufgabe. Die technische Umsetzung wurde jeweils an das betreffende Framework angepasst.

Die Ergebnisse sind jedoch als praktische Voranalyse und nicht als vollständig standardisierter Benchmark zu verstehen. Es wurde nur eine einzelne, vergleichsweise kompakte Abfrage untersucht. Deshalb lassen sich daraus keine allgemeingültigen Aussagen für jede mögliche Analyse ableiten.

Insbesondere die gemessenen Gesamtzeiten sind nur eingeschränkt aussagekräftig. Sie hängen unter anderem von Dateiformat, Kompression, Initialisierung, Caching, Speicherzustand und der konkreten Implementierung der Abfrage ab. Aussagekräftiger sind daher die Kombination aus Ladezeit, Berechnungszeit, Speicherbedarf, erfolgreicher Verarbeitung und praktischer Nutzbarkeit.

2.2 Ergebnisse nach Dateiformat

2.2.1 ZIP als ursprüngliches Eingabeformat

Die ZIP-Datei ist das bereitgestellte Ausgangsformat des Projekts. Die Heatmap zeigt, dass dieses Format vor allem für eine direkte Verarbeitung mit Pandas geeignet ist.

Für Dask und PySpark ist die ZIP-Datei als direkte Eingabe nur eingeschränkt beziehungsweise nicht zuverlässig nutzbar. Das Archiv stellt keine optimal verarbeitbare tabellarische Datenquelle für diese Frameworks dar.

Die ZIP-Datei sollte deshalb als ursprüngliche Eingabedatei erhalten bleiben. Für die weitere Verarbeitung wird sie zunächst mit Pandas eingelesen und in ein geeigneteres Arbeitsformat überführt.

2.2.2 BZ2

Die BZ2-Datei kann von mehreren Frameworks verarbeitet werden und ist damit flexibler als die ZIP-Datei. Trotzdem zeigen die Messergebnisse, dass BZ2 gegenüber Parquet nicht die günstigste Arbeitsgrundlage darstellt.

BZ2 eignet sich insbesondere als komprimiertes Austausch- oder Archivformat. Für wiederholte Analysen ist Parquet jedoch besser geeignet, da die Daten spaltenorientiert gespeichert werden und von Pandas sowie PySpark effizient eingelesen werden können.

2.2.3 Parquet

Parquet erweist sich als besonders geeignetes Arbeitsformat. Es kann sowohl mit Pandas als auch mit PySpark verwendet werden und unterstützt damit den Wechsel zwischen einfacher lokaler Analyse und komplexeren Verarbeitungsschritten.

Für das Projekt bietet Parquet insbesondere folgende Vorteile:

- spaltenorientierte Speicherung,
- effiziente Kompression,
- geringerer Speicherbedarf,
- gute Eignung für wiederholte Abfragen,
- Nutzung durch Pandas und PySpark,
- geeignete Grundlage für vorbereitete Analyse- und Dashboard-Daten.

Parquet wird daher als zentrales internes Datenformat empfohlen.

2.3 Qualitativer Vergleich der Frameworks

2.3.1 Pandas – einfache lokale Datenanalyse

Pandas eignet sich besonders für die Bereinigung, Vorbereitung und Analyse von Daten auf einem lokalen Rechner.

Stärken:

- einfache und übersichtliche Bedienung,
- sehr gute Integration in Python und Jupyter,
- schnelle Umsetzung kleiner und mittlerer Analysen,
- gute Eignung für Datenbereinigung und explorative Auswertung,
- direkte Weiterverwendung der Ergebnisse in Visualisierungen und Dashboards.

Einschränkungen:

- Verarbeitung erfolgt grundsätzlich im lokalen Python-Prozess,
- große Datenmengen können durch den verfügbaren Arbeitsspeicher begrenzt werden,
- komplexe oder sehr umfangreiche Verarbeitungsschritte sind weniger gut skalierbar.

2.3.2 Dask – parallele Verarbeitung mit Pandas-ähnlicher Syntax

Dask bietet eine Pandas-nahe Arbeitsweise und kann Daten auf mehrere Partitionen verteilen. Dadurch ist es grundsätzlich für größere lokale Datenmengen und parallele Python-Verarbeitung geeignet.

In dieser Untersuchung zeigte sich jedoch, dass die Eignung stark vom Dateiformat und von dessen Teilbarkeit abhängt. Für die vorliegende Projektstruktur bietet Dask deshalb keinen entscheidenden Vorteil gegenüber der Kombination aus Pandas und PySpark.

Dask wird daher nicht als zentrale Technologie des Hauptprojekts empfohlen, kann aber für spezielle lokale Parallelverarbeitung weiterhin eingesetzt werden.

2.3.3 PySpark – komplexe und skalierbare Verarbeitung

PySpark eignet sich insbesondere für komplexe Abfragen, größere Datenmengen und Verarbeitungsszenarien, die von paralleler oder verteilter Ausführung profitieren.

Stärken:

- Verarbeitung großer Datenmengen,
- parallele und verteilte Ausführung,
- SQL-Unterstützung,
- Verarbeitung mehrerer Dateien und Datenquellen,
- gute Erweiterbarkeit für spätere Cluster-Umgebungen,
- effiziente Verarbeitung von Parquet-Dateien.

Einschränkungen:

- höherer Einrichtungsaufwand,
- zusätzliche Java- und JVM-Abhängigkeiten,
- höherer konzeptioneller Aufwand als bei Pandas,
- für einfache lokale Analysen häufig überdimensioniert.

PySpark sollte daher gezielt für komplexere Verarbeitungsschritte eingesetzt werden und nicht für jede einfache Datenoperation.

2.4 Empfohlene Architektur für das Projekt

Für das Projekt wird folgende Verarbeitungskette empfohlen:

2.4.1 Schritt 1: Ausgangsdaten einlesen

Die ursprüngliche ZIP-Datei bleibt als Eingabedatei erhalten. Sie wird mit Pandas eingelesen, da Pandas für dieses konkrete Ausgangsformat die zuverlässigste und einfachste Verarbeitung ermöglicht.

2.4.2 Schritt 2: Daten bereinigen und vorbereiten

Einfache Verarbeitungsschritte wie

- Bereinigung fehlerhafter oder fehlender Werte,
- Anpassung von Datentypen,
- Entfernung nicht benötigter Spalten,
- grundlegende Filterungen,
- Umbenennung und Strukturierung von Spalten

werden mit Pandas durchgeführt.

2.4.3 Schritt 3: Bereinigten Datensatz als Parquet speichern

Nach der Vorbereitung wird der bereinigte Datensatz als Parquet-Datei gespeichert. Diese Datei bildet die zentrale Arbeitsgrundlage für die weiteren Analysen.

Die zusätzlich erzeugten CSV-, BZ2- und Parquet-Testdateien dieses Evaluierungsnotebooks dienen nur dem Framework-Vergleich. Sie werden nach Abschluss der Untersuchung wieder entfernt beziehungsweise nicht als Projektartefakte weiterverwendet. Im eigentlichen Projekt bleiben somit die ursprünglichen Eingangsdaten und die bewusst erzeugten Ergebnisdateien erhalten.

2.4.4 Schritt 4: Komplexe Analysen mit PySpark durchführen

Komplexere Abfragen und umfangreichere Verarbeitungsschritte werden auf der bereinigten Parquet-Datei mit PySpark ausgeführt.

PySpark bietet hierfür insbesondere bei mehreren Verarbeitungsschritten, komplexeren Aggregationen oder größeren Datenmengen Vorteile. Die Verwendung von Parquet vermeidet dabei eine erneute Verarbeitung der ursprünglichen ZIP-Datei.

2.4.5 Schritt 5: Ergebnisse wieder als Pandas-kompatible Parquet-Datei speichern

Die Ergebnisse der PySpark-Analysen werden anschließend in einer Form gespeichert, die mit Pandas eingelesen werden kann. Dadurch können sie später im Visualisierungs- oder Dashboard-Notebook weiterverwendet werden.

Damit entsteht folgende klare Trennung:

- **Pandas:** Bereinigung, Vorbereitung und einfache Analysen,
- **PySpark:** komplexe und potenziell umfangreiche Analysen,
- **Parquet:** gemeinsames und effizientes Übergabeformat,
- **Pandas beziehungsweise Dashboard:** finale Visualisierung der Ergebnisse.

2.5 Abschließende Empfehlung

Für das Hauptprojekt wird eine Kombination aus Pandas, PySpark und Parquet empfohlen.

Pandas übernimmt die leicht verständlichen und lokal gut ausführbaren Arbeitsschritte. Der bereinigte Datensatz wird anschließend als Parquet-Datei gespeichert. PySpark wird nur dort eingesetzt, wo komplexere Abfragen, umfangreichere Datenmengen oder eine spätere Skalierung einen zusätzlichen Vorteil bieten.

Die Analyseergebnisse werden wiederum als Pandas-kompatible Parquet-Dateien gespeichert. Dadurch können sie ohne erneute aufwendige Verarbeitung direkt für Tabellen, Diagramme und das Dashboard verwendet werden.

Diese Architektur verbindet eine einfache Bedienung im frühen Teil der Datenpipeline mit einer skalierbaren Verarbeitung für anspruchsvollere Analysen. Gleichzeitig bleibt die gesamte Verarbeitung nachvollziehbar, modular und für die spätere Visualisierung geeignet.

[44]:
```
# sicherheitshalber

# Dask
client.close()
```

```
# Spark
spark.catalog.clearCache()
spark.stop()
```

[45]:
```
# Dateien Löschen
files_to_delete = [
    ORIGINAL_DS_BZ2_PATH,
    ORIGINAL_DS_PARQUET_PATH,
    ORIGINAL_DS_CSV_PATH,
]

for path in files_to_delete:
    path = Path(path)

    if path.is_file():
        relative_path = get_project_relative_path(path)
        path.unlink()
        print(f"Datei gelöscht: {relative_path}")

    elif path.exists():
        relative_path = get_project_relative_path(path)
        print(
            f"Nicht gelöscht - kein reguläres Dateiobjekt: "
            f"{relative_path}"
        )

    else:
        relative_path = get_project_relative_path(path)
        print(f"Nicht vorhanden: {relative_path}")
```

```
Datei gelöscht: data/DBtrainrides.bz2
Datei gelöscht: data/DBtrainrides.parquet
Datei gelöscht: data/DBtrainrides.csv
```